



RÉPUBLIQUE DU BÉNIN

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE

UNIVERSITÉ NATIONALE DES SCIENCES, TECHNOLOGIES,
INGÉNIERIE ET MATHÉMATIQUES



ÉCOLE NATIONALE SUPÉRIEURE DE GÉNIE MATHÉMATIQUE ET
MODÉLISATION

N° attribué par la bibliothèque

--	--	--	--	--	--	--	--	--	--

MÉMOIRE DE FIN DE FORMATION

POUR L'OBTENTION DU DIPLÔME D'INGÉNIEUR DE CONCEPTION

Spécialité : **Mathématiques Appliquées et Modélisation**

Option : **Informatique, Logistique et Recherche Opérationnelle (ILRO)**

**Sécurité des applications web : Application des CPG (Code Property Graphs)
et des GNN (Graph Neural Networks) pour la détection automatique des
vulnérabilités XSS et SQLi**

Présenté par :

Sèdofi Hope LANMAYIKPOHOUE

Sous la supervision de :

Dr M. Abel KONNON

*Enseignant-Chercheur à l'INSTI / UNSTIM
Maître-Conférence des Universités du CAMES*

Sous l'encadrement de :

Dr Ing M. Géraud AZEHOUN PAZOU

*Enseignant-Chercheur à l'INSPEI / UNSTIM
Maître-Assistant des Universités du CAMES*

Dr Ing John R. O. AOGA

*Enseignant-Chercheur en Science de Données
et en Intelligence Artificielle*

Dr Ing Emery K. ASSOGBA

Enseignant-Chercheur à l'IFRI / UAC

Soutenu à Abomey, le 24 octobre 2024

Composition du jury

Président : **Prof Guy DEGLA**, Enseignant-Chercheur à l'IMSP/UAC

Membres : 1 – **Dr (MC) M. Abel KONNON**, Enseignant-Chercheur à l'INSTI/UNSTIM

2 – **Dr (MA) Ing M. Géraud AZEHOUN PAZOU**, Enseignant-Chercheur à l'INSPEI/UNSTIM

3 – **Dr (MA) Valérie HONTINFINDE**, Enseignant-Chercheur à l'ENSGMM/UNSTIM

Année académique : 2023 - 2024

4ème promotion

DÉDICACE

“

*À mon père et à ma mère, dont le soutien indéfectible et
l'encouragement constant m'ont accompagné tout au long de
mes années d'études.*

*Puissent-ils y voir l'expression de ma profonde gratitude et
reconnaissance.*

”

Sèdofi Hope LANMAYIKPOHOUE

REMERCIEMENTS

Avant de présenter ce travail, il est important pour moi d'exprimer toute ma reconnaissance envers ceux qui, de près ou de loin, m'ont soutenu et ont contribué à la réalisation de ce mémoire. À chacun d'entre vous, je tiens à dire un grand merci.

Je commence par remercier **Dr (MC) Aliou MOUSSA DJIBRIL**, Coordonnateur de l'École, ainsi que l'ensemble du corps administratif et enseignant de l'INSPEI et de l'ENSGMM pour avoir porté un vif intérêt à notre formation pendant ces cinq dernières années.

Je remercie également mon superviseur **Dr M. Abel KONNON** pour sa supervision attentive et ses retours constructifs notamment ses corrections apportées à ce travail, ainsi que mon encadrant **Dr Ing M. Géraud AZEHOUN PAZOU** pour son accompagnement et son encadrement rigoureux tout au long de cette aventure.

Mes remerciements vont aussi au **Dr Ing John R. O. AOGA** et au **Dr Ing Emery K. ASSOGBA**, mes tuteurs, pour leur accompagnement, leurs précieux conseils et leur soutien tout au long de ce travail. Leurs orientations et conseils avisés ont affiné ma réflexion, ont guidé mes recherches et ont été d'une grande aide dans l'avancement de ce projet.

Un merci tout particulier à mon condisciple **Marcellin E. T. OGOUNGBE** qui m'a permis d'accéder à son compte sur le serveur de l'IMSP. Sans son aide précieuse, en exploitant leur centre de calcul pour le traitement de mes données, ce projet n'aurait pu aboutir.

Je souhaite exprimer toute ma reconnaissance à ma tante **Pulchérie K. LANMAYIKPO-HOUE** et à son époux **Sylvain A. FADODE** pour leur hospitalité, leur bienveillance et leur soutien durant toute ma période de recherche et de rédaction de ce mémoire.

Je n'oublie pas mes **camarades de promotion** pour le chemin parcouru ensemble et pour les moments de partage et de solidarité qui ont marqué ces années d'études.

À **tous mes amis** qui m'ont encouragé et soutenu moralement, je vous dis merci du fond du cœur.

Ma reconnaissance s'étend à l'entreprise **MIFY** qui m'a accueilli pour mon stage de fin de formation.

Et pour terminer, je tiens à exprimer toute ma gratitude aux **membres du jury** pour le temps qu'ils consacreront à l'évaluation de ce travail. L'attention qu'ils y porteront et leur regard critique sont pour moi un grand honneur.

RÉSUMÉ

Cette recherche explore l'application des Code Property Graphs (CPG) et des Graph Neural Networks (GNN) pour la détection automatique des vulnérabilités XSS (Cross-Site Scripting) et SQLi (SQL injection) dans les applications web en se concentrant sur le langage PHP. À travers le développement d'un modèle hybride, combinant les architectures GCN et GATv2, des performances élevées ont été atteintes avec une accuracy (exactitude) de 97.86% pour la détection des vulnérabilités XSS et de 93.10% pour les vulnérabilités SQLi. L'étude met en avant l'efficacité des méthodes basées sur les graphes pour capturer la structure et la sémantique du code source, ce qui permet une identification précise des failles de sécurité. Les résultats soulignent également les défis techniques rencontrés, notamment en matière de temps de calcul pour l'extraction des graphes et la formation des embeddings. Enfin, quelques pistes d'amélioration sont suggérées, telles que la capacité à identifier les lignes vulnérables, ainsi que l'extension du modèle à d'autres types de vulnérabilités et de langages de programmation.

Mots-clés : Sécurité des applications web, détection de vulnérabilités, graphes de propriétés de code, réseaux de neurones de graphes, injections XSS et SQL, apprentissage profond.

ABSTRACT

This research investigates the application of Code Property Graphs (CPG) and Graph Neural Networks (GNN) for the automatic detection of XSS (Cross-Site Scripting) and SQLi (SQL injection) vulnerabilities in web applications, focusing on the PHP language. Through the development of a hybrid model combining GCN and GATv2 architectures, high performance was achieved, with an accuracy of 97.86% for detecting XSS vulnerabilities and 93.10% for SQLi vulnerabilities. The study highlights the effectiveness of graph-based approaches in capturing the structure and semantics of source code, enabling precise identification of security flaws. The results also underscore the technical challenges encountered, particularly regarding computation time for graph extraction and embedding formation. Finally, improvement perspectives are proposed, such as identifying vulnerable lines and extending the model to other types of vulnerabilities and programming languages.

Keywords: Web application security, vulnerability detection, graph neural networks, code property graphs, XSS and SQL injections, deep learning.

TABLE DES MATIÈRES

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Table des matières	v
Table des figures	viii
Liste des tableaux	ix
Acronymes et abréviations	x
Introduction	1
1 Contexte et justification	1
2 But de la recherche	2
3 Problème et questions de recherche	2
4 Objectifs	2
5 Structure du présent document	2
1 Revue de littérature	4
1.1 Introduction	4
1.2 Introduction à la sécurité des applications web	4
1.2.1 Importance de la sécurité web	4
1.2.1.1 Protection des données sensibles	4
1.2.1.2 Maintien de la confiance des utilisateurs	5
1.2.1.3 Protection contre les pertes financières	5
1.2.2 Vulnérabilités web les plus courantes (Top 10 OWASP 2021)	5
1.2.2.1 Broken Access Control	6
1.2.2.2 Cryptographic Failures	6
1.2.2.3 Injection	6
1.2.2.4 Insecure Design	6
1.2.2.5 Security Misconfiguration	7
1.2.2.6 Vulnerable and Outdated Components	7

1.2.2.7	Identification and Authentication Failures	7
1.2.2.8	Software and Data Integrity Failures	7
1.2.2.9	Security Logging and Monitoring Failures	7
1.2.2.10	Server-Side Request Forgery	8
1.3	Injections XSS et SQL	8
1.3.1	Injection XSS	8
1.3.2	Injection SQL	10
1.4	Méthodes de détection des vulnérabilités	12
1.4.1	Analyse statique	12
1.4.2	Analyse dynamique	12
1.4.3	Modèles de ML/DL	13
1.4.4	Audit manuel	13
1.5	Introduction aux Code Property Graphs (CPG)	13
1.5.1	Composition du CPG	14
1.5.1.1	Abstract Syntax Tree	14
1.5.1.2	Control Flow Graph	15
1.5.1.3	Program Dependence Graph	15
1.5.2	Construction du CPG	16
1.6	Introduction aux Graph Neural Networks (GNN)	17
1.6.1	Concept de GNN	17
1.6.2	Quelques types de GNN	18
1.6.2.1	Graph Convolutional Networks (GCN)	18
1.6.2.2	Graph Attention Networks (GAT)	18
1.6.2.3	Gated Graph Neural Networks (GGNN)	19
1.6.2.4	Graph SAMple and aggreGatE (GraphSAGE)	19
1.6.2.5	Graph Isomorphism Networks (GIN)	19
1.7	Synthèse des travaux exploitant les GNN pour la détection de vulnérabilités dans le code source	19
1.8	Conclusion	21
2	Matériel et méthodes	22
2.1	Introduction	22
2.2	Environnement expérimental	22
2.2.1	Infrastructure matérielle	22
2.2.2	Logiciels et bibliothèques	23
2.3	Collecte et préparation des données	23
2.4	Représentations des données	25
2.4.1	Quelques types de représentations de données	25
2.4.2	Extraction des Code Property Graphs (CPG)	26
2.5	Formation des embeddings (vectorisation)	26
2.6	Architecture du modèle	28
2.6.1	Description des composants	28
2.6.1.1	Couche GCN	28
2.6.1.2	Couche GAT(v2)	29
2.6.1.3	Jumping Knowledge	30
2.6.1.4	Mécanisme de pooling global	30
2.6.2	Description du modèle	31
2.7	Évaluation du modèle	32

2.7.1	Processus d'entraînement	32
2.7.2	Métriques d'évaluation	33
2.8	Conclusion	34
3	Résultats et discussion	35
3.1	Introduction	35
3.2	Présentation des résultats	35
3.2.1	Résultats de la détection des vulnérabilités XSS	35
3.2.2	Résultats de la détection des vulnérabilités SQLi	39
3.3	Discussion	42
3.4	Conclusion	44
	Conclusion générale et perspectives	45
	Références bibliographiques	47
	Bibliographie	47
	Webographie	50

TABLE DES FIGURES

1.1	Fuite de données Twitter	5
1.2	Code vulnérable au XSS	9
1.3	Exemple d'une vulnérabilité XSS exploitée par un attaquant	9
1.4	Requête SQL non sécurisée	11
1.5	Payload exploitant une requête SQL non sécurisée	11
1.6	Illustration d'une attaque par injection SQL	11
1.7	Exemple de code PHP	14
1.8	Exemple d'AST	14
1.9	Exemple de CFG	15
1.10	Exemple de PDG	16
1.11	Processus de message passing	17
2.1	Formation des embeddings	27
2.2	Architecture du modèle	31
3.1	Courbe ROC (ensemble de données XSS)	36
3.2	Courbe d'apprentissage (ensemble de données XSS)	37
3.3	Courbe d'exactitude (ensemble de données XSS)	37
3.4	Histogramme de prédictions (ensemble de données XSS)	38
3.5	Matrice de confusion (ensemble de données XSS)	38
3.6	Courbe d'exactitude (ensemble de données SQLi)	40
3.7	Courbe d'apprentissage (ensemble de données SQLi)	40
3.8	Courbe ROC (ensemble de données SQLi)	41
3.9	Histogramme de prédictions (ensemble de données SQLi)	41
3.10	Matrice de confusion (ensemble de données SQLi)	42

LISTE DES TABLEAUX

2.1	Logiciels et bibliothèques utilisés	23
2.2	Distribution initiale des données pour CWE-79 (XSS)	24
2.3	Distribution initiale des données pour CWE-89 (SQLi)	24
2.4	Distribution finale des données après sous-échantillonnage	24
3.1	Hyperparamètres utilisés pour l'entraînement	35
3.2	Métriques de performance (ensemble de données XSS)	36
3.3	Métriques de performance (ensemble de données SQLi)	39
3.4	Comparaison des performances du modèle entre XSS et SQLi	42
3.5	Performances du modèle sur différentes architectures GNN	43
3.6	Temps de calcul de l'implémentation du modèle	43

ACRONYMES ET ABRÉVIATIONS

AST	Abstract Syntax Tree
AUC	Area Under the Curve
CFG	Control Flow Graph
CNN	Convolutional Neural Network
CPG	Code Property Graph
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DL	Deep Learning
GAT	Graph Attention Network
GATv2	Graph Attention Network, version 2
GCN	Graph Convolutional Network
GGNN	Gated Graph Neural Network
GGRN	Gated Graph Recurrent Network
GIN	Graph Isomorphism Network
GNN	Graph Neural Network
GraphSAGE	Graph SAmple and aggreGatE
GRU	Gated Recurrent Unit
JK	Jumping Knowledge
ML	Machine Learning
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
PDG	Program Dependence Graph
ReLU	Rectified Linear Unit
RF	Random Forest
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SARD	Software Assurance Reference Dataset
SEGNN	Sequential Graph Neural Network
SQL	Structured Query Language
SQLi	SQL injection
SVM	Support Vector Machine
XSS	Cross-Site Scripting

L'École Nationale Supérieure de Génie Mathématique et Modélisation (ENSGMM) n'entend donner ni approbation ni improbation aux opinions émises dans ce document. Les raisonnements et arguments reviennent donc à l'auteur.

INTRODUCTION

1 Contexte et justification

La sécurité des applications web est devenue une grande préoccupation dans le domaine des technologies de l'information. Avec la montée en puissance des services en ligne, les applications web jouent un rôle très important dans la simplification de nos activités quotidiennes, qu'il s'agisse de transactions bancaires, de services e-commerce ou d'interactions sur les réseaux sociaux. Cependant, cette omniprésence expose ces applications à des vulnérabilités majeures, notamment les attaques par injection XSS (Cross-Site Scripting) et par injection SQL (Structured Query Language), qui restent parmi les plus persistantes et dangereuses.

Ces vulnérabilités sont particulièrement fréquentes dans les applications web développées en PHP. Ce langage, largement utilisé pour créer des sites web dynamiques et des plateformes interactives, est souvent ciblé en raison de son intégration directe avec des bases de données et de son utilisation généralisée dans des systèmes critiques comme les CMS (Content Management Systems) et les boutiques en ligne.

Selon des études récentes [1], environ 75.5% des sites web utilisent PHP côté serveur, ce qui en fait l'un des langages backend les plus répandus. Cette prévalence rend PHP particulièrement vulnérable aux failles comme XSS et SQLi qui sont exploitées pour compromettre les systèmes, accéder à des données sensibles ou nuire à l'intégrité des applications. Par ailleurs, les vulnérabilités XSS et SQLi (SQL injection) ont constamment figuré parmi les plus dangereuses dans le Top 25 du classement établi par CWE (Common Weakness Enumeration) [2]. Depuis 2022, elles occupent respectivement la deuxième et la troisième place parmi les vulnérabilités web les plus dangereuses. Ces positions élevées indiquent la persistance et la dangerosité de ces deux des plus anciennes failles de sécurité web qui ne disparaissent clairement pas [3]. Les attaquants peuvent les exploiter pour accéder à des données sensibles, compromettre l'intégrité des informations ou prendre le contrôle des systèmes, entraînant ainsi des pertes financières, des violations de données et des atteintes à la réputation des entreprises affectées.

Pour faire face à ces menaces croissantes, il est impératif de mettre en place des méthodes efficaces pour détecter et prévenir ces vulnérabilités dès les premières phases de développement. Les méthodes traditionnelles, qui reposent souvent sur des analyses statiques et dynamiques, présentent des limites en termes de précision et de couverture. L'intégration de Code Property Graphs (CPG) et de Graph Neural Networks (GNN) représente ainsi une solution parfaite pour améliorer la détection de ces vulnérabilités. Les CPG permettent de modéliser la structure et la sémantique du code source sous forme de graphe, tandis que les GNN utilisent ces représentations pour apprendre des modèles complexes et identifier les failles de sécurité.

2 But de la recherche

Ce mémoire vise à explorer et à développer un modèle basé sur les CPG et les GNN pour la détection automatique des vulnérabilités XSS et SQLi dans les applications web développées en PHP, puisque la plupart des outils de détection existants se concentrent principalement sur les langages de programmation comme C et C++. En exploitant les capacités de ces deux technologies, cette étude vise à améliorer la précision et l'efficacité de la détection des vulnérabilités XSS et SQLi tout en réduisant le temps et les efforts nécessaires pour effectuer des audits de sécurité manuels.

3 Problème et questions de recherche

Le principal problème abordé dans cette recherche est de savoir comment améliorer la détection des vulnérabilités XSS et SQLi dans les applications web en utilisant des méthodes basées sur l'analyse de graphes et l'apprentissage profond. Les questions de recherche qui en découlent sont les suivantes :

- **QR1** : *Dans quelle mesure une méthode combinant les CPG et les GNN peut-elle améliorer la détection des vulnérabilités XSS et SQLi ?*
- **QR2** : *Quelles sont les performances du modèle proposé en termes d'exactitude, de précision, de rappel et d'autres métriques d'évaluation ?*
- **QR3** : *Quels sont les défis et les limitations de la méthode proposée et comment peuvent-ils être surmontés ?*

4 Objectifs

L'objectif général de cette recherche est de développer une méthode automatisée et efficace pour la détection des vulnérabilités XSS et SQLi dans les applications web PHP. Spécifiquement, ce travail vise à :

- Concevoir un modèle utilisant les CPG et les GNN pour détecter les vulnérabilités XSS et SQLi dans le code PHP.
- Évaluer les performances du modèle en termes d'exactitude, de précision, de rappel et d'autres métriques.
- Optimiser les paramètres du modèle pour maximiser la précision et l'efficacité.
- Évaluer les performances du modèle par rapport aux modèles GNN les plus courants.
- Identifier et analyser les défis et les limitations du modèle développé et proposer des solutions pour les surmonter.

5 Structure du présent document

Le reste de ce document est structuré comme suit : Le **Chapitre 1** propose une revue de littérature approfondie sur la sécurité des applications web, les vulnérabilités XSS et SQLi, les Code Property Graphs (CPG) et les Graph Neural Networks (GNN). Le **Chapitre 2** décrit les matériels et méthodes utilisés, y compris l'architecture du modèle développé, les ensembles de données utilisés et les techniques d'évaluation des modèles. Le **Chapitre 3** analyse les performances du modèle et les implications des résultats obtenus. Enfin, le

5. Structure du présent document

dernier chapitre résume les conclusions de la recherche et propose des perspectives pour recherches futures.

REVUE DE LITTÉRATURE

1.1 Introduction

Dans ce chapitre, nous allons aborder les concepts théoriques essentiels liés à la sécurité des applications web et aux vulnérabilités XSS et SQLi. Nous présenterons ensuite les différentes méthodes de détection de vulnérabilités pour enfin finir sur la clarification des concepts de CPG et de GNN.

1.2 Introduction à la sécurité des applications web

1.2.1 Importance de la sécurité web

La sécurité web est un enjeu important pour les applications web d'aujourd'hui. Avec le développement fulgurant des services en ligne, les risques pesant sur les données et les systèmes informatiques n'ont jamais été aussi élevés et complexes. La sécurité web est primordiale pour plusieurs raisons essentielles, notamment la protection des données sensibles, le maintien de la confiance des utilisateurs et la protection contre les pertes financières.

1.2.1.1 Protection des données sensibles

L'un des aspects les plus critiques de la sécurité des applications web réside dans la protection des données sensibles. Ces applications sont amenées à collecter et à traiter une grande quantité d'informations critiques telles que des données personnelles, des informations financières et des données de santé. Une faille de sécurité peut entraîner la divulgation de ces informations, causant ainsi des dommages significatifs tant aux individus qu'aux organisations.

Prenons l'exemple de la violation de données d'Equifax (2017) qui a exposé les informations personnelles de 147 millions de personnes, y compris des numéros de sécurité sociale, des dates de naissance et des adresses [4]. De même, au début de 2023, une fuite de données a révélé les informations de plus de 200 millions d'utilisateurs de Twitter sur un forum de hackers bien connu [5]. Les données compromises comprenaient des adresses e-mail, des noms complets, des noms d'utilisateur, le nombre d'abonnés et les dates de création de compte des utilisateurs (figure 1.1). En janvier 2024, une autre violation a touché Trello, avec la divulgation de plus de 15 millions d'adresses e-mail, de noms complets et de noms d'utilisateur, mises en vente sur un forum en ligne [6]. Ces incidents soulignent l'importance

1.2. Introduction à la sécurité des applications web

de mettre en place des mesures de sécurité robustes pour protéger les données sensibles contre tout accès non autorisé.

```
Email: [redacted] - Name: Joe Osullivan - ScreenName: JoeOsullivan2 - Followers: 263 - Created At: [redacted]
Email: [redacted] - Name: Pandora - ScreenName: FemaleWitticism - Followers: 68 - Created At: Sat Apr 21
Email: [redacted] - Name: Eileen - ScreenName: EileenEmh66 - Followers: 3 - Created At: Thu Dec 19 12:3
Email: [redacted] - Name: Sarah Thurlow ♥ * EU - ScreenName: sjthurlow - Followers: 950 - Created
Email: [redacted] - Name: manishg4 - ScreenName: manishg4 - Followers: 332 - Created At: Sat Mar 21 17
Email: [redacted] - Name: Victoria McIntyre - ScreenName: VictoriaMcInty1 - Followers: 3 - Cre
Email: [redacted] - Name: amanda foster - ScreenName: mandyquinlivan - Followers: 0 - Created At
Email: [redacted] - Name: Roy Hughes - ScreenName: royjhughes - Followers: 3 - Created At: Sat
Email: [redacted] - Name: lesley stevens - ScreenName: lesleystevens55 - Followers: 5 - Creat
Email: [redacted] - Name: I'm Sooooo Confused - ScreenName: myConfusedLook - Followers: 0 - Crea
Email: [redacted] - Name: Rachael Carroll - ScreenName: rachaellox - Followers: 0 - Created At: Mon
```

Figure 1.1. Fuite de données Twitter (BleepingComputer [5])

1.2.1.2 Maintien de la confiance des utilisateurs

La confiance des utilisateurs est un élément essentiel pour le succès d'une application web. Les utilisateurs doivent avoir le sentiment que leurs informations sont en sécurité pour continuer à utiliser les services en ligne. Lorsque cette confiance est brisée, les conséquences peuvent être désastreuses pour l'entreprise.

Les entreprises qui subissent des violations de sécurité voient souvent leur réputation gravement ternie. La violation de données de Yahoo en 2013-2014, qui a affecté 3 milliards de comptes, en est un exemple frappant. Outre les pertes financières, cet incident a gravement nui à la réputation de l'entreprise [7]. Une étude menée par Ponemon Institute (2017) a montré que 65% des consommateurs ont perdu confiance dans une organisation suite à une ou plusieurs violations de données [8]. Cette même étude révèle que les consommateurs privilégient les entreprises qui mettent en œuvre des mesures de sécurité robustes et qui prennent au sérieux la protection de leurs informations personnelles. La sécurité des applications web joue donc un rôle non négligeable dans le maintien de cette confiance.

1.2.1.3 Protection contre les pertes financières

Les conséquences financières des failles de sécurité peuvent être énormes, allant des pertes directes liées aux cyberattaques aux coûts indirects tels que le vol de fonds, les coûts de remédiation, la perte de revenus due à l'interruption des activités.

Selon une étude menée par IBM (2020), le coût moyen d'une violation de données a atteint 3.86 millions de dollars en 2020 [9]. Ce chiffre montre l'ampleur des dommages financiers potentiels causés par une faille de sécurité. De plus, selon le rapport de Verizon (2021) sur les enquêtes sur les violations de données, 39% des violations de données ont impliqué des applications web [10]. Les failles dans les applications web représentent donc un risque majeur. Après une violation de données, les clients peuvent hésiter à s'offrir les services de l'entreprise victime, entraînant une baisse des revenus. La perte de clients et la baisse de la fidélité peuvent ainsi avoir un impact financier à long terme sur l'entreprise, ce qui peut dépasser les coûts immédiats de la violation.

1.2.2 Vulnérabilités web les plus courantes (Top 10 OWASP 2021)

Les applications web sont confrontées à de nombreuses vulnérabilités potentielles. L'Open Web Application Security Project (OWASP) [11] publie régulièrement une liste des dix

principaux risques de sécurité pour ces applications qui est largement reconnue comme un guide essentiel pour comprendre les menaces les plus critiques. Examinons les vulnérabilités les plus courantes selon le Top 10 OWASP 2021.

1.2.2.1 Broken Access Control (Contrôle d'accès défaillant)

Le contrôle d'accès défaillant se situe en tête de liste des vulnérabilités web les plus critiques. Selon OWASP (2021), 94% des applications testées ont présenté une forme de contrôle d'accès défaillant [11]. Cette faille survient lorsque les systèmes ne parviennent pas à restreindre correctement les actions des utilisateurs authentifiés. Un attaquant pourrait exploiter ces faiblesses pour accéder, modifier ou supprimer des données auxquelles il ne devrait pas avoir accès ou pour exécuter des fonctions administratives sans autorisation. Les manifestations courantes de cette vulnérabilité incluent l'élévation de privilèges, la manipulation d'URL pour contourner les restrictions et l'exploitation de failles dans les API qui ne vérifient pas correctement les autorisations au niveau des objets.

1.2.2.2 Cryptographic Failures (Défaillances cryptographiques)

Les défaillances cryptographiques, anciennement désignées sous le terme d'exposition de données sensibles (Sensitive Data Exposure), occupent la deuxième place dans le classement OWASP. Cette catégorie englobe les vulnérabilités liées à la protection insuffisante des données sensibles. Les erreurs fréquentes incluent la transmission de données sensibles en clair, l'utilisation d'algorithmes cryptographiques faibles ou obsolètes, ainsi qu'une gestion inappropriée des clés. Ces failles peuvent avoir des conséquences graves en compromettant à la fois la confidentialité et l'intégrité des données. Par exemple, des informations critiques telles que les données personnelles des utilisateurs ou les secrets d'entreprise peuvent être exposées à des attaquants.

1.2.2.3 Injection

L'injection, bien que rétrogradée de la première place, reste une menace sérieuse pour la sécurité des applications web. Cette vulnérabilité se produit lorsqu'une application accepte des données non fiables qui peuvent modifier le comportement des requêtes envoyées à un interpréteur. Un attaquant exploitant avec succès une vulnérabilité d'injection pourrait voler ou manipuler des données, gagner un contrôle non autorisé sur le système ou même causer un déni de service. Les types d'injection les plus courants sont l'injection de commande, l'injection XML, l'injection LDAP, l'injection SQL et l'injection XSS. Ces deux dernières, qui font l'objet de cette recherche, sont présentées dans la section 1.3.

1.2.2.4 Insecure Design (Conception non sécurisée)

Nouvelle entrée dans le Top 10 de 2021, la conception non sécurisée met en avant les risques associés aux défauts de conception des applications. Cette catégorie souligne l'importance d'incorporer la sécurité dès les premières étapes de la conception. Elle se concentre sur les failles au niveau de l'architecture de l'application plutôt que sur les erreurs d'implémentation. Parmi les problèmes courants, on trouve l'absence de limites sur l'utilisation des ressources, le manque de séparation des privilèges et l'absence de mécanismes de récupération après une attaque. Ces vulnérabilités sont souvent difficiles à corriger une fois l'application déployée.

1.2.2.5 Security Misconfiguration (Mauvaise configuration de sécurité)

Les erreurs de configuration de sécurité représentent une source fréquente de vulnérabilités. Cette catégorie englobe plusieurs types d'erreurs de configuration qui peuvent créer des failles de sécurité involontaires. Cela peut être des services ou des ports inutiles laissés ouverts, des comptes et mots de passe par défaut non modifiés, des permissions excessives accordées ou des paramètres de sécurité mal configurés. Ces erreurs peuvent offrir à un attaquant un accès non autorisé ou exposer des fonctionnalités et des données sensibles.

1.2.2.6 Vulnerable and Outdated Components (Composants vulnérables et obsolètes)

L'utilisation de composants vulnérables ou obsolètes peut introduire des failles de sécurité importantes dans une application web. En effet, ces applications dépendent souvent de nombreuses bibliothèques et frameworks externes. Si ces composants ne sont pas régulièrement mis à jour, ils peuvent contenir des vulnérabilités connues qui vont se répercuter sur la sécurité globale de l'application. Les attaquants peuvent alors exploiter ces failles pour compromettre la sécurité du système.

1.2.2.7 Identification and Authentication Failures (Identification et authentification de mauvaise qualité)

Les défaillances d'identification et d'authentification regroupent diverses faiblesses dans les mécanismes utilisés pour vérifier l'identité des utilisateurs. Ces vulnérabilités peuvent inclure des problèmes tels que la possibilité d'attaques par force brute, une mauvaise gestion des sessions ou un stockage non sécurisé des mots de passe. Les conséquences de ces failles peuvent être graves, allant de l'usurpation d'identité à l'accès non autorisé à des comptes privilégiés. Pour se protéger contre ces risques, il est essentiel que les applications mettent en œuvre des contrôles d'authentification solides comme l'authentification multifacteur et la limitation des tentatives de connexion.

1.2.2.8 Software and Data Integrity Failures (Manque d'intégrité des données et du logiciel)

Cette nouvelle catégorie dans le Top 10 de 2021 met l'accent sur les problèmes liés à l'intégrité du code et des données, en particulier dans le cadre des mises à jour logicielles et des pipelines CI/CD. Ces vulnérabilités peuvent permettre l'injection de code malveillant ou la manipulation des données de l'application. Parmi les exemples figurent les mises à jour de logiciels effectuées sans vérification d'intégrité ou l'utilisation de bibliothèques provenant de sources non fiables.

1.2.2.9 Security Logging and Monitoring Failures (Défaillances des systèmes de contrôle et de journalisation)

Une journalisation et une surveillance insuffisantes peuvent nuire à la détection rapide des activités malveillantes et compliquer la réponse aux incidents. Cette catégorie inclut des problèmes tels que l'absence de journalisation des événements critiques, un stockage inapproprié des journaux ou le manque d'alertes pour les activités suspectes. Sans une surveillance et une journalisation adéquates, une organisation risque de ne pas détecter une attaque en cours ou de ne pas être en mesure de réaliser une analyse post-incident efficace.

1.2.2.10 Server-Side Request Forgery (Falsification de requête côté serveur)

La Server-Side Request Forgery (SSRF) est une vulnérabilité qui permet à un attaquant de manipuler une application afin d'envoyer des requêtes vers des destinations non prévues, souvent en contournant les contrôles d'accès réseau. Les attaques SSRF peuvent être exploitées pour accéder à des services internes qui sont normalement inaccessibles, contourner des pare-feu ou exfiltrer des données via des requêtes DNS.

1.3 Injections XSS et SQL

1.3.1 Injection XSS

Le Cross-Site Scripting (XSS) est une vulnérabilité de sécurité présente dans les applications web qui permet à un attaquant d'injecter des scripts malveillants dans des pages web visualisées par d'autres utilisateurs. Une attaque XSS se produit lorsqu'un attaquant parvient à injecter du code malveillant, généralement du JavaScript, dans une page web consultée par d'autres utilisateurs. Ce code s'exécute alors dans le navigateur de la victime et permet à l'attaquant d'effectuer diverses activités malveillantes telles que transférer des informations privées, envoyer des requêtes malveillantes à un site Web au nom de la victime ou prendre le contrôle de la machine de la victime [12].

Les attaques XSS se classifient principalement en trois catégories :

- **XSS réfléchi (non persistant) :** Le script malveillant est injecté via une requête HTTP et est renvoyé immédiatement dans la réponse du serveur. Par exemple, un attaquant pourrait envoyer un lien à une victime et si celle-ci clique sur le lien, le script s'exécute dans son navigateur.
- **XSS stocké (persistant) :** Le code malveillant est stocké sur le serveur (par exemple, dans une base de données) et est exécuté chaque fois qu'un utilisateur accède à la page vulnérable. Cela peut se produire dans des forums, des systèmes de commentaires ou des applications de messagerie. Le XSS persistant est considéré comme plus dangereux.
- **XSS basé sur le DOM :** Dans ce type d'attaque, le script malveillant modifie le Document Object Model (DOM) de la page web, permettant à l'attaquant de manipuler le contenu affiché à l'utilisateur. Il se produit entièrement dans le navigateur du client.

Le fonctionnement d'une attaque XSS repose sur l'exploitation d'une validation insuffisante des entrées utilisateur par l'application web. Lorsqu'une application intègre des données non fiables dans son contenu sans les valider ou sans les échapper correctement, elle devient vulnérable aux attaques XSS. Le navigateur de la victime, incapable de distinguer le code légitime du code malveillant, exécute alors tout le script présent sur la page.

1.3. Injections XSS et SQL

```
<script>
  function afficherMessage() {
    var nom = new URLSearchParams(window.location.search).get('nom');
    document.getElementById('message').innerHTML = 'Bonjour, ' + nom + '!';
  }
</script>
<body onload="afficherMessage()">
  <div id="message"></div>
</body>
```

Figure 1.2. Code vulnérable au XSS

Dans l'exemple de la figure 1.2, si un attaquant manipule le paramètre `nom` dans l'URL pour inclure du code JavaScript malveillant, comme `http://exemple.com/page?nom=<script>alert("XSS")</script>`, le script s'exécutera lorsque la page sera chargée. Une application sécurisée devrait échapper correctement les caractères spéciaux ou utiliser des méthodes plus sûres pour manipuler le DOM [13].

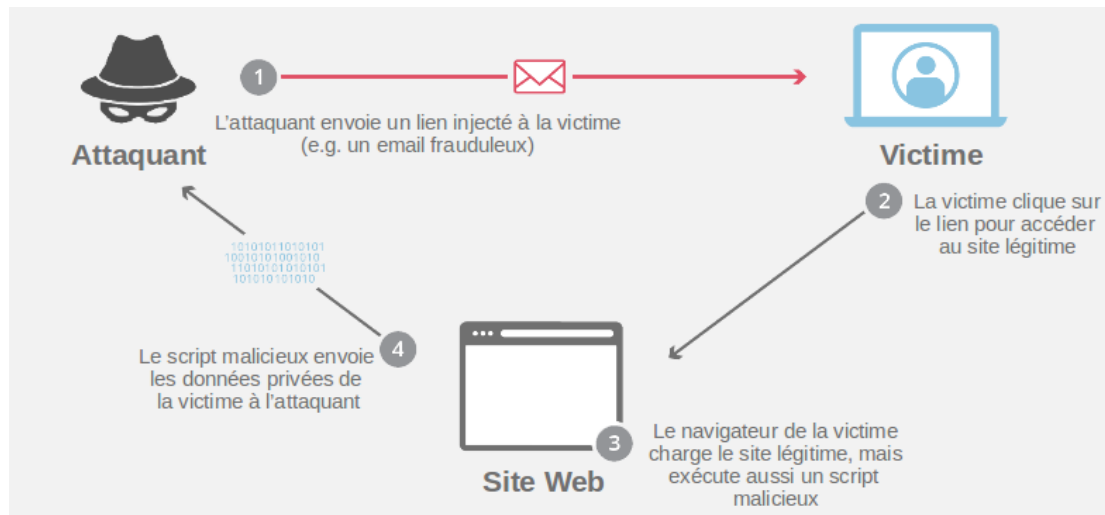


Figure 1.3. Exemple d'une vulnérabilité XSS exploitée par un attaquant [14]

Les attaques XSS sont particulièrement dangereuses en raison de leur polyvalence et de leur potentiel de dommages. Elles peuvent être utilisées pour voler des cookies de session, permettant ainsi à l'attaquant d'usurper l'identité de la victime sur le site web ciblé. De plus, elles peuvent être exploitées pour effectuer du phishing ciblé en modifiant le contenu de la page web pour tromper l'utilisateur et l'inciter à révéler des informations sensibles. Dans certains cas, les attaques XSS peuvent même être utilisées pour propager des malwares (logiciels malveillants) ou prendre le contrôle du navigateur de la victime.

L'un des cas d'attaques XSS les plus célèbres est celui du *ver Samy* créé par Samy Kamkar et qui a frappé la plateforme MySpace en 2005. Exploitant une vulnérabilité XSS persistante, ce ver ¹ a infecté plus d'un million de profils en moins de 24 heures en ajoutant automatiquement l'attaquant comme ami et affichant le message *"but most of all, Samy is my hero"* sur les profils compromis [15].

¹ Un ver informatique est un programme malveillant capable de se répliquer et de se propager automatiquement d'un système à un autre, sans nécessiter l'intervention humaine.

Un autre exemple est l'attaque XSS qui a touché Twitter en 2010. Cette vulnérabilité permettait l'exécution automatique de code JavaScript lorsqu'un utilisateur passait simplement sa souris sur un tweet malveillant. L'attaque a été exploitée pour rediriger les utilisateurs vers des sites pornographiques et propager des messages politiques en affectant des comptes de haut profil, y compris celui de l'épouse de l'ancien Premier ministre britannique [16].

Ces incidents soulignent l'importance de la protection contre les attaques XSS dans le développement d'applications web. Ils démontrent également la rapidité avec laquelle une vulnérabilité XSS peut être exploitée à grande échelle en causant des dommages considérables à la réputation d'une entreprise et en compromettant la sécurité de nombreux utilisateurs.

1.3.2 Injection SQL

L'injection SQL (SQLi) est une vulnérabilité particulièrement dangereuse et répandue qui cible les applications web qui utilisent des bases de données relationnelles. Elle se produit lorsqu'un attaquant parvient à insérer des instructions SQL malveillantes dans les requêtes de l'application qui résultent de l'exploitation d'une validation insuffisante des entrées utilisateur. Cette attaque permet à l'attaquant de lire, modifier ou supprimer des données sensibles de la base de données, contourner les mécanismes d'authentification, exécuter des commandes d'administration sur la base de données, injecter des commandes au système d'exploitation et dans certains cas, prendre le contrôle total de la base de données [17].

Il existe plusieurs types d'injections SQL, chacun exploitant différentes faiblesses dans la conception ou l'implémentation de l'application.

- **Injections SQL en aveugle (Blind SQL Injection) :** Elles sont utilisées lorsque l'application ne renvoie pas directement les résultats de la requête, obligeant l'attaquant à déduire les informations par des méthodes indirectes.
- **Injections SQL basées sur le temps (Time-Based SQL Injection) :** Elles exploitent les délais de réponse du serveur pour extraire des informations.
- **Injections SQL de second ordre (Second-Order SQL Injection) :** Elles impliquent l'insertion de code malveillant qui n'est pas immédiatement exécuté, mais stocké pour une utilisation ultérieure.
- **Injections SQL hors bande (Out-of-band SQL Injection) :** Elles utilisent des canaux alternatifs pour extraire les données comme le DNS ou les connexions HTTP [18].

Le fonctionnement d'une attaque par injection SQL repose sur la manipulation des requêtes SQL envoyées à la base de données. Pour illustrer ce processus, considérons l'exemple suivant d'une application web utilisant une requête SQL non sécurisée pour authentifier un utilisateur :

1.3. Injections XSS et SQL

```
<?php
//...
$username = $_POST['username'];
$password = $_POST['password'];
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = mysqli_query($connection, $query);
//...
?>
```

Figure 1.4. Requête SQL non sécurisée

Dans cet exemple, l'entrée de l'utilisateur n'est pas échappée ni validée. Alors, si un attaquant entre comme nom d'utilisateur `admin' --` et laisse le champ du mot de passe vide, la requête SQL résultante serait :

```
SELECT * FROM users WHERE username = 'admin' -- ' AND password = ''
```

Figure 1.5. Payload exploitant une requête SQL non sécurisée

Le `'--'` en SQL commente le reste de la ligne, ce qui fait que la vérification du mot de passe est ignorée. L'attaquant pourrait ainsi se connecter en tant qu'administrateur sans connaître le mot de passe.

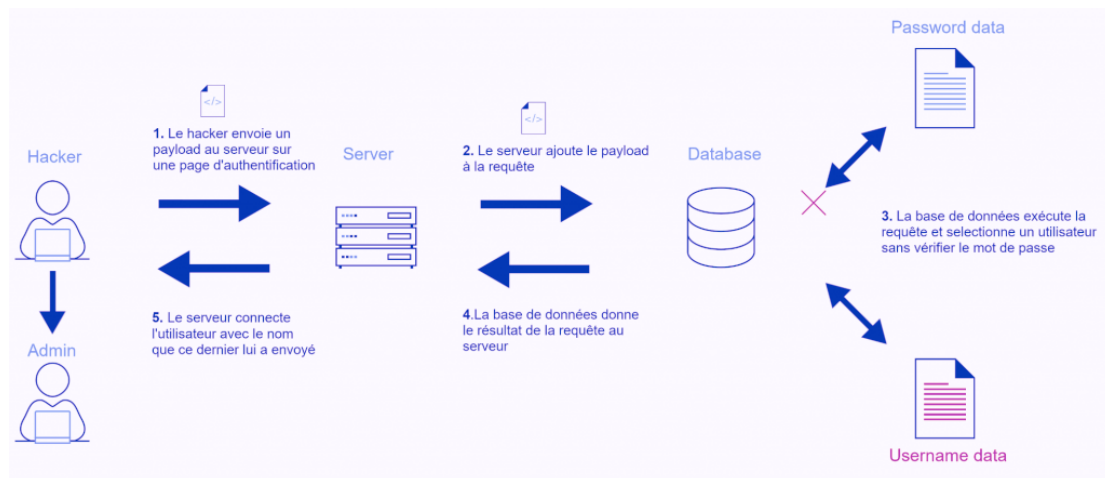


Figure 1.6. Illustration d'une attaque par injection SQL [19]

Les injections SQL sont dangereuses en raison de leur potentiel de dommages considérables. Elles peuvent permettre à un attaquant d'accéder à des données sensibles telles que des informations personnelles, des numéros de carte de crédit ou des secrets d'entreprise. De plus, elles peuvent être utilisées pour modifier ou supprimer des données critiques, ce qui altère les enregistrements de la base de données et compromet ainsi son intégrité. Dans certains cas, les injections SQL peuvent même mener à une prise de contrôle complète du serveur hébergeant la base de données, donnant la possibilité d'exploiter d'autres failles [20].

L'une des violations de données par injection SQL les plus marquantes est celle de Heartland Payment Systems en 2008. Cette attaque a permis à des hackers d'accéder à pas moins de 100 millions de numéros de cartes de crédit en exploitant une vulnérabilité SQLi [21]. Les

1.4. Méthodes de détection des vulnérabilités

conséquences ont été désastreuses, entraînant des pertes financières considérables et une atteinte à la réputation de l'entreprise.

Un autre incident est l'attaque contre Sony Pictures en 2011. Les attaquants ont exploité une vulnérabilité d'injection SQL pour accéder à la base de données contenant les informations personnelles de millions d'utilisateurs telles que des noms, adresses, dates de naissance et mots de passe. Cette violation a causé des pertes financières importantes et a gravement terni la réputation de l'entreprise [22].

Un exemple supplémentaire est l'attaque contre TJX Companies en 2007 où des hackers ont exploité une injection SQL pour accéder aux systèmes de point de vente de l'entreprise, ce qui leur a permis de voler les informations de plus de 45 millions de cartes de crédit et de débit. Cette brèche a coûté à l'entreprise plus de 250 millions de dollars en règlements et amendes [23].

En 2017, une vulnérabilité SQLi a également été exploitée lors de l'attaque contre Equifax, l'une des plus grandes agences de crédit aux États-Unis. Les attaquants ont pu accéder à des données sensibles de 147 millions de personnes, y compris des numéros de sécurité sociale et des informations de carte de crédit. Cet incident a entraîné des poursuites judiciaires et des amendes considérables pour l'entreprise, soulignant ainsi l'importance de la sécurité des données [24].

Ces événements montrent à quel point il est essentiel de se protéger contre les injections SQL lors du développement des applications web. Ils illustrent les énormes dégâts qu'une seule vulnérabilité peut provoquer en touchant non seulement l'entreprise visée, mais aussi des millions d'utilisateurs sans lien avec l'incident.

1.4 Méthodes de détection des vulnérabilités

Plusieurs méthodes peuvent être utilisées pour la détection des vulnérabilités. Elles peuvent être classées en plusieurs catégories notamment l'analyse statique, l'analyse dynamique, les méthodes basées sur l'apprentissage automatique (ML) et l'apprentissage profond (DL) et l'audit manuel. Chacune de ces méthodes présente des avantages et des inconvénients et leur efficacité dépend souvent du contexte d'application.

1.4.1 Analyse statique

L'analyse statique examine le code source ou le code compilé d'une application sans l'exécuter. Cette méthode peut détecter une variété de vulnérabilités potentielles à un stade précoce du cycle de développement. Elle fonctionne en analysant la structure du code, le flux de données et le flux de contrôle pour identifier des motifs susceptibles d'indiquer des failles de sécurité [25]. L'analyse statique présente l'avantage de pouvoir être effectuée tôt dès les premières étapes de développement et de couvrir potentiellement tout le code, y compris les chemins rarement exécutés. Elle peut cependant générer un nombre élevé de faux positifs et certaines vulnérabilités, notamment celles dépendant de l'exécution, peuvent être difficiles à détecter par cette méthode.

1.4.2 Analyse dynamique

L'analyse dynamique, quant à elle, implique l'exécution de l'application dans un environnement contrôlé et l'observation de son comportement face à diverses entrées. Cette méthode

peut détecter des vulnérabilités qui ne sont apparentes que lors de l'exécution du code [26]. Elle permet de détecter des vulnérabilités dans le contexte d'exécution réel de l'application et génère généralement moins de faux positifs que l'analyse statique. Elle est particulièrement efficace pour identifier des problèmes de validation des entrées, de contrôle d'accès, gestion des erreurs [27], de gestion de session et autres vulnérabilités runtime. Cependant, l'analyse dynamique peut ne pas couvrir tous les chemins d'exécution possibles, peut requérir beaucoup de temps et de ressources et nécessiter un environnement de test approprié.

1.4.3 Modèles de ML/DL

Les méthodes basées sur l'apprentissage automatique (ML) et l'apprentissage profond (DL) ont récemment gagné en popularité pour la détection de vulnérabilités. Ces méthodes utilisent des algorithmes d'apprentissage pour identifier des motifs et des anomalies pouvant indiquer des vulnérabilités [28]. Leur avantage est de détecter des vulnérabilités complexes ou précédemment inconnues et de s'améliorer au fil du temps avec plus de données. Les modèles de ML/DL peuvent combiner des caractéristiques de l'analyse statique et dynamique. Parmi ces modèles, on trouve les réseaux de neurones convolutifs (CNN) [29], les réseaux de neurones récurrents (RNN) [30], les forêts aléatoires (RF) [31], les machines à vecteurs de support (SVM) [31] ou encore les modèles de graphes tels que les réseaux de neurones à séquence de graphes fermés (GGNN), les réseaux de neurones à convolution de graphes (GCN) et les réseaux de neurones d'attention graphique (GAT). Cependant, ces approches nécessitent de grandes quantités de données d'entraînement de haute qualité et les résultats peuvent parfois être difficiles à interpréter ou à expliquer.

1.4.4 Audit manuel

L'audit manuel du code reste une méthode indispensable dans la détection des vulnérabilités malgré l'avancement des techniques automatisées. Cette méthode, bien que chronophage, implique l'examen minutieux du code source par des experts en sécurité qui peuvent identifier des failles subtiles que les outils automatisés pourraient manquer [27]. L'audit manuel est particulièrement efficace pour détecter des vulnérabilités liées à la logique métier et des failles complexes qui nécessitent une compréhension approfondie du contexte de l'application. Il est souvent utilisé en complément des méthodes automatisées pour une couverture de sécurité plus complète.

1.5 Introduction aux Code Property Graphs (CPG)

Les graphes de propriétés de code (CPG) sont une représentation intermédiaire du code source qui combine plusieurs graphes d'analyse statique en une structure unifiée permettant de capturer des informations essentielles sur la structure syntaxique, le flux de contrôle et les dépendances de données et de contrôle au sein du code. Cette méthode a été introduite par Yamaguchi et al. [32] dans le cadre de la découverte de vulnérabilités dans les codes sources C et C++ et a, depuis, été appliquée à divers langages de programmation et contextes d'analyse tels que la détection de clones de code, l'analyse des surfaces d'attaque et la génération d'exploits.

1.5.1 Composition du CPG

La structure d'un CPG se compose principalement de trois graphes interconnectés : l'Abstract Syntax Tree (AST), le Control Flow Graph (CFG) et le Program Dependence Graph (PDG).

```
<?php
function foo() {
    $x = $_GET["id"];
    if(isset($x)) {
        $sql = "SELECT * FROM users WHERE id = '$x'";
        query($sql);
    }
}
```

Figure 1.7. Exemple de code PHP

1.5.1.1 Abstract Syntax Tree

L'arbre de syntaxe abstraite (AST) est une représentation hiérarchique de la structure syntaxique du code source sous forme d'arbre [33]. Il peut être défini comme un graphe orienté $G_{AST} = (V_{AST}, E_{AST}, \text{root}, \lambda_{AST})$ où V_{AST} est l'ensemble des nœuds représentant les éléments syntaxiques du programme, $E_{AST} \subseteq V_{AST} \times V_{AST}$ est l'ensemble des arêtes représentant les relations hiérarchiques entre ces éléments, $\text{root} \in V_{AST}$ est le nœud racine de l'arbre et $\lambda_{AST} : V_{AST} \rightarrow \Sigma_{AST}$ est une fonction d'étiquetage qui attribue à chaque nœud un type syntaxique (par exemple, déclaration, expression, opérateur). Chaque nœud $v_i \in V_{AST}$ représente une construction syntaxique telle qu'une expression, une instruction ou un bloc de code. La structure arborescente de l'AST permet une traversée efficace et une analyse détaillée de la syntaxe du programme. Les relations entre les nœuds sont généralement des relations parent-enfant, illustrant comment les différentes parties du code sont imbriquées les unes dans les autres. Par exemple, un nœud représentant une instruction conditionnelle aurait des enfants représentant la condition et les blocs de code pour les branches if et else. Cette représentation facilite des analyses telles que la vérification de types, la détection de motifs syntaxiques spécifiques et sert de base pour de nombreuses transformations de code. À titre d'illustration, la figure 1.8 montre l'arborescence syntaxique abstraite de l'exemple de la figure 1.7.

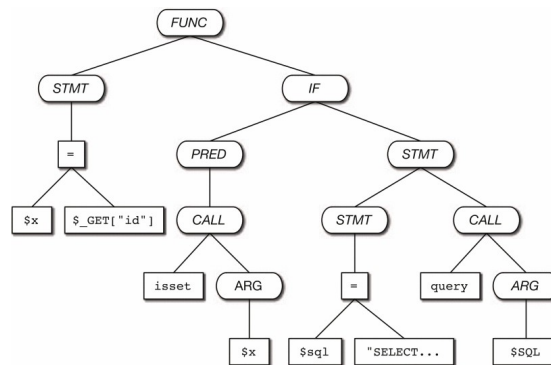


Figure 1.8. AST du code de la figure 1.7 [34]

1.5.1.2 Control Flow Graph

Le graphe de flux de contrôle (CFG) modélise la structure de contrôle d'un programme en représentant tous les chemins d'exécution possibles [35]. Un CFG peut être défini comme un graphe orienté $G_{CFG} = (V_{CFG}, E_{CFG}, \text{entry}, \text{exit}, \lambda_{CFG})$ où V_{CFG} est l'ensemble des nœuds représentant des blocs d'instructions ou des segments de code exécutés de manière séquentielle, $E_{CFG} \subseteq V_{CFG} \times V_{CFG}$ est l'ensemble des arêtes représentant les transitions de contrôle possibles entre les blocs, $\text{entry} \in V_{CFG}$ est le nœud d'entrée unique du graphe, $\text{exit} \in V_{CFG}$ est le nœud de sortie (qui peut être multiple dans certaines représentations) et $\lambda_{CFG} : E_{CFG} \rightarrow \Sigma_{CFG} = \{\text{"true"}, \text{"false"}, \text{"unconditional"}\}$ est une fonction d'étiquetage des arêtes indiquant les conditions d'association. Chaque nœud $v_i \in V_{CFG}$ correspond à un bloc de code, tandis que chaque arête $(v_i, v_j) \in E_{CFG}$ indique un chemin possible que l'exécution du programme peut suivre tel que les transitions conditionnelles ou inconditionnelles entre les blocs. Le CFG est particulièrement utile pour analyser le chemin d'exécution d'un programme et identifier les points de divergence ou de convergence dans le flux d'exécution, ce qui est essentiel pour des analyses telles que l'identification de chemins potentiellement dangereux qui pourraient mener à des états vulnérables du programme. La figure 1.9 illustre le graphique du flux de contrôle pour l'exemple de la figure 1.7 : Le flux de contrôle est principalement linéaire, à l'exception des deux arêtes provenant de l'instruction conditionnelle `isset($x)`. Selon que cette instruction est évaluée à `true` ou à `false`, le contrôle peut soit être transféré à la première instruction dans le `if`, soit l'exécution de la fonction peut se terminer, ce qui est modélisé par l'arête vers le nœud `exit` [34]. Kronjee et al. ont utilisé les CFG pour la détection de vulnérabilités XSS et SQLi [36].

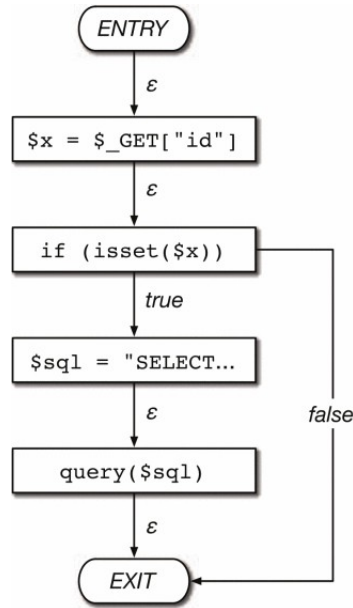


Figure 1.9. CFG du code de la figure 1.7 [34]

1.5.1.3 Program Dependence Graph

Le graphe de dépendances du programme (PDG) capture les dépendances de données et de contrôle au sein d'un programme [37]. Un PDG peut également être représenté comme un graphe orienté $G_{PDG} = (V_{PDG}, E_{data} \cup E_{control}, \lambda_{PDG})$ où V_{PDG} est l'ensemble des nœuds représentant les instructions du programme, $E_{data} \subseteq V_{PDG} \times V_{PDG}$ est l'ensemble des arêtes

1.5. Introduction aux Code Property Graphs (CPG)

de dépendance de données, $E_{\text{control}} \subseteq V_{PDG} \times V_{PDG}$ est l'ensemble des arêtes de dépendance de contrôle et $\lambda_{PDG} : E_{\text{data}} \cup E_{\text{control}} \rightarrow \Sigma_{PDG} = \{\text{"data"}, \text{"control"}\}$ est une fonction d'étiquetage qui distingue les types de dépendances. Les dépendances de données existent entre deux instructions si l'une utilise une variable définie ou modifiée par l'autre, elles montrent comment les valeurs produites par une instruction sont utilisées par une autre. Les dépendances de contrôle, quant à elles, relient une instruction de contrôle (comme une condition) aux instructions dont l'exécution dépend de cette décision. Cette représentation est utile pour l'analyse de flux de données, l'optimisation de code (comme l'élimination de code mort) et la détection de vulnérabilités liées à la manipulation incorrecte des données. Par exemple, dans l'analyse de taint, le PDG permet de suivre la propagation de données non fiables à travers le programme, identifiant ainsi les points où ces données pourraient être utilisées de manière dangereuse. À titre d'exemple, la figure 1.10 montre le graphe de dépendance du programme pour l'exemple de la figure 1.7, où les arêtes étiquetées avec D désignent les arêtes de dépendance de données et les arêtes étiquetées avec C désignent les arêtes de dépendance de contrôle [34].

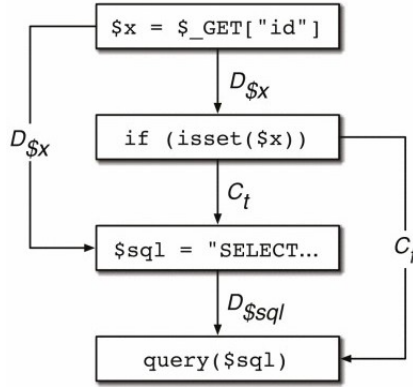


Figure 1.10. PDG du code de la figure 1.7 [34]

1.5.2 Construction du CPG

Le graphe de propriétés de code (CPG) est une structure unifiée qui intègre l'AST, le CFG et le PDG en un seul graphe multicouche. Formellement, un CPG peut être défini comme un graphe $G = (V, E, \lambda)$ où

- $V = V_{AST} \cup V_{CFG} \cup V_{PDG} = V_{AST}$ car les nœuds CFG et PDG sont des sous-ensembles des nœuds AST [32].
- $E = E_{AST} \cup E_{CFG} \cup E_{\text{data}} \cup E_{\text{control}}$
- $\lambda = \lambda_{AST} \cup \lambda_{CFG} \cup \lambda_{PDG}$

La structure résultante est un graphe orienté, étiqueté par des arêtes et attribué par des nœuds où chaque nœud peut avoir plusieurs relations avec d'autres nœuds, ce qui en fait un multigraphe. Cette structure unifiée permet une navigation fluide entre les différentes représentations du code et facilite les analyses complexes qui nécessitent la prise en compte simultanée de la syntaxe, du flux de contrôle et des dépendances. Dans la détection de vulnérabilités, le CPG permet de combiner l'analyse syntaxique (via l'AST) pour identifier les appels de fonction potentiellement dangereux, l'analyse de flux de contrôle (via le CFG) pour déterminer les chemins d'exécution menant à ces appels et l'analyse de dépendances (via le PDG) pour tracer la provenance des données utilisées dans ces appels. La capacité à

représenter des relations complexes entre les constructions de code est ce qui rend les CPG particulièrement puissants pour l'analyse de vulnérabilités et la recherche de motifs dans le code.

Plusieurs travaux de recherche [32, 38, 39, 40, 41, 42, 43] ont exploité cette méthode pour améliorer l'analyse de sécurité du code source.

1.6 Introduction aux Graph Neural Networks (GNN)

1.6.1 Concept de GNN

Les réseaux de neurones en graphes (GNN) représentent une classe de réseaux de neurones conçus pour opérer directement sur des structures de données sous forme de graphes. Les GNN constituent ainsi un prolongement des réseaux de neurones conventionnels issus du Deep Learning qui sont utilisés pour traiter des données structurées [44]. Ces modèles étendent les principes de l'apprentissage profond aux données non euclidiennes pour opérer directement sur des structures de graphes, permettant ainsi de capturer et d'exploiter les relations complexes entre les entités représentées par les nœuds et les arêtes d'un graphe [45].

Le concept fondamental des GNN repose sur l'idée de l'apprentissage de représentations vectorielles (embeddings) pour chaque nœud du graphe. Ces représentations sont apprises de manière à capturer non seulement les caractéristiques intrinsèques du nœud, mais aussi les informations structurelles de son voisinage et par extension, la structure globale du graphe. Ce processus, souvent appelé *message passing* ("propagation de message" ou "agrégation de voisinage"), implique l'agrégation itérative d'informations provenant des nœuds voisins pour mettre à jour la représentation de chaque nœud et permet ainsi aux GNN de modéliser efficacement les relations complexes et les interdépendances au sein des données structurées en graphe.



Figure 1.11. *Processus de message passing* [46]

Soit $G(V, E)$ un graphe, où V représente l'ensemble des nœuds (ou sommets) et E l'ensemble des arêtes (ou liens) entre les nœuds. Chaque nœud v_i a une représentation (ou un vecteur de caractéristiques) h_i^0 et chaque arête (v_i, v_j) peut également avoir une représentation (ou un vecteur de caractéristiques) e_{ij} .

Étant donné que les GNN fonctionnent en plusieurs couches de propagation de messages, à chaque couche l , l'algorithme de GNN met à jour les caractéristiques des nœuds en combinant leurs caractéristiques actuelles avec les caractéristiques de leurs voisins.

1.6. Introduction aux Graph Neural Networks (GNN)

Pour un nœud v_i , la mise à jour des caractéristiques à la couche $l + 1$ est généralement formulée comme suit :

$$h_i^{l+1} = f(h_i^l, \text{AGG}(\{h_j^l \mid (v_j, v_i) \in E\})) \quad (1.1)$$

où :

- AGG est une fonction d'agrégation qui combine les caractéristiques des voisins $\{h_j^l \mid (v_j, v_i) \in E\}$ du nœud v_i .
- f est une fonction qui met à jour les caractéristiques du nœud v_i en tenant compte de ses propres caractéristiques et des messages agrégés de ses voisins.

Cette formulation permet aux GNN de capturer des dépendances à longue distance dans le graphe à travers des itérations successives. La fonction d'agrégation AGG peut prendre diverses formes allant de simples fonctions courantes (somme, moyenne, maximum, concaténation) à des mécanismes d'attention plus complexes.

1.6.2 Quelques types de GNN

Il existe plusieurs architectures de GNN, chacune ayant ses propres caractéristiques et conçue pour traiter des aspects spécifiques des données graphiques ou pour résoudre des types particuliers de problèmes. Voici un aperçu des principaux types de GNN.

1.6.2.1 Graph Convolutional Networks (GCN)

Les réseaux de neurones à convolution de graphes (GCN) [47] sont parmi les architectures de GNN les plus couramment utilisées. Ils adaptent le concept de convolution, déjà bien établi dans les réseaux de neurones convolutifs (CNN) pour le traitement d'images, aux données structurées sous forme de graphes. Les GCN fonctionnent en agrégeant de manière itérative les informations des nœuds voisins. À chaque couche, un nœud met à jour sa représentation en combinant sa propre représentation avec une moyenne pondérée de celles de ses voisins. Cette méthode permet aux GCN d'être à la fois efficaces en termes temps de calcul et capables de généraliser à des graphes non vus lors de l'entraînement. Les GCN ont été appliqués avec succès dans divers domaines, notamment la classification de documents dans des réseaux de citation, la prédiction de liens dans les réseaux sociaux et la classification de protéines dans les réseaux d'interaction protéine-protéine.

1.6.2.2 Graph Attention Networks (GAT)

Les Graph Attention Networks (GAT) [48, 49] introduisent un mécanisme d'attention dans les GNN. Au lieu d'agréger l'information des voisins de manière uniforme comme dans les GCN, les GAT apprennent des coefficients d'attention pour chaque paire de nœuds connectés. Ces coefficients déterminent l'importance relative de chaque voisin dans la mise à jour de la représentation d'un nœud. Cela permet aux GAT d'attribuer différents poids d'importance à différents voisins lors de l'agrégation de l'information. De plus, le mécanisme d'attention peut être parallélisé sur plusieurs têtes d'attention, ce qui augmente la capacité du modèle à capturer différents types de relations entre les nœuds. Les GAT sont utiles lorsque l'importance des connexions dans le graphe n'est pas uniforme ou est inconnue a priori.

1.6.2.3 Gated Graph Neural Networks (GGNN)

Les Gated Graph Neural Networks (GGNN) [50] utilisent des unités de mise à jour basées sur les Gated Recurrent Units (GRU) pour propager l'information à travers le graphe. Les GGNN traitent la propagation de l'information dans le graphe comme un processus récurrent en utilisant des GRU pour contrôler le flux d'information entre les nœuds au fil du temps. Ce mécanisme permet aux GGNN de capturer des dépendances à long terme dans le graphe et de traiter des graphes avec des caractéristiques d'arêtes. Le processus de mise à jour dans les GGNN implique plusieurs étapes contrôlées par des portes, similaires à celles utilisées dans les réseaux de neurones récurrents (RNN). Cela permet au modèle de décider quelle information conserver ou oublier à chaque étape de propagation.

1.6.2.4 Graph Sample and aggreGatE (GraphSAGE)

GraphSAGE [51] est une méthode inductive conçue pour l'apprentissage sur de grands graphes et permettant de générer des embeddings pour des nœuds non vus pendant l'entraînement. Au lieu d'utiliser toute la structure du graphe comme les GCN, GraphSAGE échantillonne un nombre fixe de voisins pour chaque nœud et apprend une fonction d'agrégation qui peut être appliquée à n'importe quel nœud. Cette méthode le rend plus scalable et capable de gérer des graphes dynamiques ou partiellement observés. GraphSAGE fonctionne en échantillonnant et agrégeant les caractéristiques des nœuds voisins pour générer des embeddings pour chaque nœud, puis en combinant cette information avec les propres caractéristiques du nœud. Cela permet à GraphSAGE de générer des embeddings pour des nœuds non vus pendant l'entraînement, ce qui le rend spécialement utile pour des applications impliquant des graphes dynamiques ou des graphes de très grande taille où l'apprentissage d'embeddings pour tous les nœuds n'est pas pratique.

1.6.2.5 Graph Isomorphism Networks (GIN)

Les Graph Isomorphism Networks (GIN) [52] sont conçus pour maximiser la puissance discriminative des GNN en s'inspirant du test d'isomorphisme de Weisfeiler-Lehman (WL) [53]. GIN vise à créer des représentations de nœuds qui sont aussi puissantes que possible pour distinguer différentes structures de graphe. Il utilise une fonction d'agrégation basée sur la somme qui est injective (préserve l'unicité), ce qui lui permet de capturer des structures de graphe complexes. GIN fonctionne en appliquant itérativement des transformations non linéaires et des agrégations sur les nœuds du graphe en utilisant un perceptron multicouche (MLP) pour traiter les informations agrégées. Cette méthode permet à GIN d'être aussi puissant que le test WL pour distinguer les structures de graphe et le rend efficace pour des tâches nécessitant une compréhension fine de la topologie du graphe. GIN a montré d'excellentes performances dans des tâches de classification de graphes, notamment dans l'analyse de molécules et la prédiction de propriétés moléculaires.

1.7 Synthèse des travaux exploitant les GNN pour la détection de vulnérabilités dans le code source

L'application des réseaux de neurones de graphes (GNN) à la détection de vulnérabilités dans le code source offre plusieurs avantages clés. Les GNN permettent de modéliser le code comme un graphe où les nœuds représentent les éléments du code tels que les variables et

fonctions et les arêtes indiquent les relations entre eux telles que les appels de fonction et les dépendances de données. Ils capturent également les dépendances à longue distance essentielles pour identifier les vulnérabilités résultant d'interactions complexes entre différentes parties du code grâce à leur mécanisme de propagation de message. En outre, les GNN sont capables d'apprendre des représentations vectorielles riches pour chaque élément du code en intégrant les informations syntaxiques et sémantiques. Plusieurs travaux de recherche ont déjà exploré l'utilisation des GNN pour détecter de vulnérabilités.

L'une des premières contributions majeures dans ce domaine est le travail de Zhou et al. (2019) qui ont proposé Devign, un modèle basé sur les GNN pour la détection de vulnérabilités dans le code source C/C++. Ils ont été les premiers à utiliser un GNN pour les tâches de détection de vulnérabilité de code [38]. Leur méthode se distingue par une représentation composite du code source sous forme de graphe de propriétés de code, intégrant l'arbre syntaxique abstrait, le graphe de flux de contrôle, le graphe de flux de données et la séquence naturelle du code. L'architecture de Devign utilise des GNN multicouches notamment basées sur des GGRN (Gated Graph Recurrent Networks) avec des unités récurrentes à portes (GRU) pour agréger l'information des nœuds voisins, complétée par un module de convolution pour extraire des caractéristiques pertinentes au niveau du graphe entier. Les résultats de l'évaluation de leur modèle sur quatre grands projets open-source en C démontrent la supériorité de Devign par rapport aux méthodes existantes basées sur l'apprentissage automatique, avec un gain moyen de 10.51% en précision et 8.68% en score F1. De plus, Devign surpasse de loin les analyseurs statiques sur des jeux de données déséquilibrés et atteint une précision de 74.11% sur 40 CVE (Common Vulnerabilities and Exposures) récentes [54].

AI4VA [42], un pipeline développé par Suneja et al. (2020), convertit d'abord le code source en un graphe de propriétés de code (CPG) en préservant ainsi la structure et la sémantique du code. Ce graphe est ensuite vectorisé et traité par un GGNN pour apprendre automatiquement les signatures des vulnérabilités. Les résultats empiriques démontrent que cette méthode surpasse les analyseurs statiques ainsi que d'autres modèles d'apprentissage profond basés les CNN et les RNN sur plusieurs jeux de données.

Cheng et al. (2021) ont introduit DeepWukong, un système de détection statique de vulnérabilités basé sur les GNN qui se concentre sur l'analyse de code interprocédurale pour les programmes C/C++ [55]. DeepWukong construit un graphe de code interprocédural qui capture les relations entre les fonctions, ce qui permet de détecter des vulnérabilités qui s'étendent sur plusieurs fonctions. Le modèle utilise une architecture composée de GCN et de GAT avec un mécanisme de pooling global pour encoder les fragments de code dans une représentation à faible dimension en préservant les flux de contrôle et de données ainsi que les informations en langage naturel. Les expériences ont montré l'efficacité de DeepWukong sur un vaste corpus de 105428 programmes réels, surpassant les détecteurs de vulnérabilités statiques et les méthodes d'apprentissage profond existantes.

Dans une autre étude, Ahmed et al. (2023) proposent un réseau de neurones de graphes séquentiel (SEGNN) pour l'identification précoce des vulnérabilités dans le code source [41]. Leur méthode s'appuie sur une représentation du code sous forme de graphes de flux de contrôle (CFG) et utilise des embeddings Word2Vec comme caractéristiques des nœuds. Le modèle SEGNN intègre un module d'apprentissage séquentiel, des couches de convolution de graphes et des mécanismes d'attention et de pooling pour apprendre diverses repré-

1.8. Conclusion

tions sémantiques du code. Les auteurs ont également constitué un nouveau jeu de données, CVEFGE, à partir de la base de données CVE comprenant des vulnérabilités authentiques du monde réel. Leurs expériences menées sur deux jeux de données et comparées à quatre méthodes de référence démontrent des performances supérieures dans la détection des vulnérabilités, avec une amélioration allant jusqu'à 20% par rapport aux méthodes existantes.

Plus récemment (avril 2024), Liu et al. [38] ont proposé Vul-LMGNN, un modèle pour la détection de vulnérabilités dans le code source qui combine les modèles de langage de code pré-entraînés, notamment CodeBERT avec les graphes de propriétés de code. Leur méthode construit un graphe de propriétés de code intégrant divers attributs du code (syntaxe, contrôle de flux et dépendances de données) et utilise un modèle de langage pré-entraîné pour extraire des caractéristiques sémantiques locales comme embeddings de nœuds. Ils appliquent un GGNN pour préserver efficacement les informations de dépendance entre les différentes propriétés. En entraînant conjointement le modèle de langage de code et les modules GGNN, Vul-LMGNN exploite les forces des deux mécanismes. CodeBERT est également utilisé comme classificateur auxiliaire permettant de combiner les résultats finaux de détection par interpolation linéaire. Évalué sur quatre jeux de données de vulnérabilités réelles, Vul-LMGNN surpasse six méthodes de pointe en termes de performances.

Ces recherches récentes mettent en évidence l'efficacité croissante des réseaux de neurones de graphes dans la détection de vulnérabilités dans le code source, en surpassant souvent les méthodes statiques et d'autres approches d'apprentissage profond. L'évolution des modèles illustre une tendance vers des architectures plus avancées qui combinent les GNN avec d'autres techniques telles que les modèles de langage pré-entraînés.

1.8 Conclusion

Cette revue de littérature a mis en lumière l'importance de la sécurité des applications web, en particulier face aux menaces que représentent les injections XSS et SQL tout en soulignant le potentiel remarquable des Code Property Graphs (CPG) et des Graph Neural Networks (GNN) dans la détection automatique de ces vulnérabilités.

MATÉRIEL ET MÉTHODES

2.1 Introduction

Dans ce chapitre, nous présentons en détail l'approche méthodologique adoptée pour la détection automatique des vulnérabilités XSS (CWE-79) [12] et SQLi (CWE-89) [56] dans les applications web. Nous décrivons le processus complet, de la collecte des données à la conception et l'entraînement d'un modèle hybride, en passant par la conversion des codes sources en graphes et leur vectorisation.

2.2 Environnement expérimental

L'environnement expérimental a joué un rôle essentiel dans la réalisation de notre étude, affectant à la fois les capacités de traitement et les choix méthodologiques. Cette section présente le matériel et les logiciels utilisés pour mener à bien notre recherche sur la détection des vulnérabilités XSS et SQLi.

2.2.1 Infrastructure matérielle

Pour nos expériences, nous avons exploité un ordinateur de l'Institut de Mathématiques et de Sciences Physiques (IMSP) et fourni par Calcul Bénin. L'infrastructure de calcul repose sur un serveur équipé des composants suivants :

Système d'exploitation	: CentOS Linux release 8.5.2111
Processeur	: Intel(R) Xeon(R) Silver 4114 CPU @ 2.20GHz
Nombre de cœurs	: 40
Mémoire RAM	: 128 GB

Cette configuration fournit une puissance de calcul considérable, idéale pour gérer de grands volumes de données et pour entraîner des modèles graphiques complexes. Avec ses 40 cœurs, elle a permis d'optimiser le parallélisme lors des étapes de prétraitement des données et de construction des graphes. De plus, la grande capacité de mémoire RAM (128 Go) a été importante pour travailler avec des ensembles de données volumineux et pour exécuter efficacement les algorithmes d'apprentissage profond sur les structures de graphes.

2.2.2 Logiciels et bibliothèques

Notre étude s’est également appuyée sur un écosystème logiciel centré autour de Python et de bibliothèques spécialisées en science de données et en apprentissage profond. Nous avons utilisé une série d’outils et de frameworks spécifiques que nous avons listés dans le tableau 2.1 avec leurs versions respectives.

Tableau 2.1. Logiciels et bibliothèques utilisés

Logiciels/Bibliothèques	Version	Description
Python	3.12.4	Langage de programmation principal pour l’implémentation de notre méthode
Numpy	2.0.1	Bibliothèque fondamentale pour le calcul scientifique en Python
Pandas	2.2.2	Utilisée pour la manipulation et l’analyse des données structurées
PyTorch	2.4.0	Framework d’apprentissage profond choisi pour sa flexibilité et ses performances
PyTorch Geometric	2.5.3	Extension de PyTorch pour l’apprentissage profond sur des graphes, essentielle pour la conception de notre modèle
Transformers	4.43.3	Bibliothèque pour l’utilisation de modèles de langage pré-entraînés, notamment pour l’embedding initial des nœuds avec CodeBERT
Seaborn	0.13.2	Utilisées pour la visualisation des données et des résultats
Matplotlib	3.9.0	
Scikit-learn	1.5.1	Employée pour diverses tâches de prétraitement des données et d’évaluation des modèles
Joern	2.0.384	Plateforme d’analyse de code utilisée pour l’extraction des Code Property Graphs (CPG) à partir des échantillons de code source

Cet environnement expérimental a non seulement facilité la gestion de certains défis techniques, mais a également eu un impact sur nos choix méthodologiques, en particulier en ce qui concerne le sous-échantillonnage de nos données.

2.3 Collecte et préparation des données

La première étape vers la création d’un modèle de détection des vulnérabilités consiste à collecter des données vulnérables et non vulnérables pertinentes pour former les modèles [27]. C’est une étape très importante car la qualité et la diversité de ces données conditionneront directement la robustesse et la capacité de généralisation des modèles qui en découleront, modèles qui devront être en mesure de détecter efficacement les failles de sécurité, quels que soient le contexte et le style de programmation. Dans le cadre de notre étude sur la détection des vulnérabilités XSS et SQLi, nous avons eu recours à trois sources de données différentes, chacune apportant sa pierre à l’édifice et permettant d’enrichir notre corpus d’apprentissage de manière complémentaire.

La première source que nous avons utilisée est le Software Assurance Reference Dataset (SARD) [57], une initiative du National Institute of Standards and Technology (NIST) des États-Unis d’Amérique. Ce projet propose une vaste collection de codes sources dont

2.3. Collecte et préparation des données

une partie couvrant diverses vulnérabilités logicielles. SARD fournit des exemples de code annotés, allant de scénarios simples à des cas plus élaborés, ce qui en fait un outil précieux pour évaluer et améliorer les outils de détection de vulnérabilités.

Pour compléter cette base, nous avons utilisé le Vulnerability Test Suite Generator (VTSG) [58], également développé par le NIST, ainsi que le PHP Vulnerability Test Suite Generator [59]. Ces outils permettent de créer dynamiquement des codes sources annotés, ce qui augmente la diversité de notre jeu de données et nous aide à explorer différentes variations de vulnérabilités.

Après avoir collecté les données provenant de nos trois sources, nous avons constitué un ensemble de données conséquent pour chaque type de vulnérabilité. Par la suite, nous avons procédé à un prétraitement qui consistait à supprimer tous les commentaires présents dans les codes sources. Cette étape a permis de nettoyer les données en ne conservant que le code exécutable, ce qui a réduit le bruit potentiel que les commentaires auraient pu introduire dans notre analyse. Cela a également aidé notre modèle à se concentrer sur les structures de code essentielles pour détecter les vulnérabilités. Après ce nettoyage, nous présentons ci-dessous la répartition initiale de nos données dans les tableaux 2.2 et 2.3.

Tableau 2.2. *Distribution initiale des données pour CWE-79 (XSS)*

Source	Safe	Unsafe	Total
PHP-VTSG	5728	4352	10080
SARD	130712	28543	159255
Total	136440	32895	169335

Tableau 2.3. *Distribution initiale des données pour CWE-89 (SQLi)*

Source	Safe	Unsafe	Total
PHP-VTSG	8640	912	9552
SARD	74374	20150	94524
VTSG	175296	109824	285120
Total	258310	130886	389196

Au total, notre ensemble de données initial comptait 558531 échantillons.

Cependant, durant la phase d’embeddings (voir section 2.5), nous avons réalisé que le traitement d’un volume aussi important de données dépassait nos capacités de calcul, ce qui entraînait des temps de traitement excessifs. Pour remédier à cette situation et améliorer notre processus d’apprentissage, nous avons décidé de procéder à un sous-échantillonnage de nos données. Cela nous a permis de maintenir un bon équilibre entre les classes vulnérables (unsafe) et non vulnérables (safe) tout en réduisant considérablement la taille de notre jeu de données. La répartition finale après ce sous-échantillonnage est présentée dans le tableau 2.4 ci-dessous.

Tableau 2.4. *Distribution finale des données après sous-échantillonnage*

Vulnérabilité	Safe	Unsafe	Total
CWE-79 (XSS)	25000	25000	50000
CWE-89 (SQLi)	23412	23412	46824

2.4 Représentations des données

2.4.1 Quelques types de représentations de données

1. **Représentation basée sur les tokens** : Cette méthode consiste à transformer le code en une séquence de tokens, chaque token représentant un élément lexical du langage (mots-clés, identificateurs, opérateurs, etc). Elle est simple à mettre en œuvre et efficace pour certaines tâches comme la détection de code dupliqué ou l'analyse de style. Cependant, cette méthode perd la structure hiérarchique du code et les relations complexes entre ses éléments.
2. **Représentation basée sur les arbres** : Ce type de représentation préserve la structure hiérarchique du code en permettant une analyse plus fine de sa syntaxe. L'AST en est l'exemple le plus courant. Les arbres sont utiles pour les analyses qui nécessitent une compréhension de la structure du programme comme la refactorisation automatique ou certaines optimisations de compilateur. Néanmoins, ils ne capturent pas explicitement les flux de contrôle et de données.
3. **Représentation basée sur les graphes** : C'est la représentation la plus riche et flexible. Les graphes peuvent encoder simultanément plusieurs aspects du code : sa structure syntaxique, son flux de contrôle, ses dépendances de données et même des informations sémantiques. Cette représentation permet de capturer des relations complexes et non linéaires dans le code, essentielles pour des analyses avancées comme la détection de vulnérabilités ou l'analyse de flux de données inter-procédurales. Les graphes peuvent prendre diverses formes telles que les graphes de flux de contrôle (CFG), les graphes de dépendance de programme (PDG) ou des représentations plus complexes combinant plusieurs types de relations comme les CPG.
4. Autres représentations :
 - **Représentations vectorielles** : Ces représentations, inspirées du traitement du langage naturel, transforment le code en vecteurs numériques denses. Elles sont particulièrement utiles pour les applications d'apprentissage automatique.
 - **Représentations basées sur les métriques** : Ces méthodes condensent le code en un ensemble de métriques numériques utiles pour l'évaluation de la qualité du code.
 - **Représentations symboliques** : Elles utilisent des formules logiques pour représenter le comportement du programme souvent utilisées dans la vérification formelle.

Pour ce projet qui vise à développer un modèle de Graph Neural Network (GNN) pour la détection de vulnérabilités, la représentation sous forme de graphes se révèle être la plus adaptée. Les GNN sont spécifiquement conçus pour travailler avec des structures de graphes, ce qui leur permet d'exploiter les relations complexes et multidimensionnelles qu'elles représentent. Cette représentation facilite la capture simultanée de la structure du code, des flux de contrôle et de données, ainsi que d'autres relations sémantiques essentielles pour l'analyse de sécurité.

Les vulnérabilités telles que XSS et SQLi, qui impliquent souvent des flux de données complexes et des interactions entre différentes parties du code, sont bien modélisées par des graphes. Cela permet à un modèle GNN d'examiner plusieurs aspects du code en même temps, ce qui aide à identifier des motifs subtils pouvant indiquer des vulnérabilités.

En optant ainsi pour une représentation basée sur les graphes, notamment en utilisant des

2.5. Formation des embeddings (vectorisation)

graphes de propriétés de code (CPG) comme base pour l'analyse, nous garantissons que le modèle GNN dispose des informations les plus complètes et pertinentes pour réaliser sa tâche avec précision et efficacité.

2.4.2 Extraction des Code Property Graphs (CPG)

Pour extraire les CPG de notre corpus de codes PHP, nous avons mis en place un processus en deux étapes en utilisant l'outil Joern [60]. Joern est une plateforme d'analyse de code statique open-source réputée pour sa capacité à générer des représentations détaillées du code sous forme de graphes de propriétés.

La première phase consiste à transformer les fichiers PHP en objets CPG binaires. Nous exploitons les fonctionnalités de Joern pour traiter chaque fichier PHP et générer un fichier binaire correspondant. Ces fichiers binaires contiennent la représentation CPG complète du code source, capturant ainsi sa structure et ses propriétés.

La seconde étape implique la conversion des objets CPG binaires en format JSON qui offre une structure lisible et facilement manipulable.

Nous nous sommes particulièrement concentrés sur l'extraction des nœuds et des arêtes des trois composantes principales (AST, CFG et PDG). Cette extraction est effectuée à deux niveaux : pour le code global et pour chaque fonction individuelle présente dans le code source.

La structure JSON résultante contient une liste détaillée des nœuds, chacun avec ses propriétés spécifiques (type de nœud, numéro de ligne, bout de code correspondant, etc) et les arêtes (entrantes et sortantes) associées.

Cette représentation JSON capture non seulement la structure syntaxique du code (via l'AST), mais également son flux de contrôle (via le CFG) et ses dépendances de données (via le PDG).

Pour rendre ce processus d'extraction moins exigeant en ressources, nous avons adopté une stratégie de traitement parallèle. Grâce à cette parallélisation, nous avons pu gérer un grand nombre de fichiers en même temps en tirant parti des capacités multi-cœurs de notre infrastructure de calcul. De plus, nous avons mis en place un système de traitement par lots, ce qui nous a permis de mieux contrôler l'utilisation des ressources et de gérer plus efficacement la charge de travail.

2.5 Formation des embeddings (vectorisation)

Les *embeddings* (vectorisations) sont une technique qui vise à transformer des entités complexes et leurs relations en vecteurs de caractéristiques denses. Elle permet de représenter des données complexes et discrètes sous forme de vecteurs continus dans un espace de faible dimension. Dans le contexte de l'analyse de graphes et de code source, deux types d'embeddings sont particulièrement pertinents : les *node embeddings* (vectorisation de nœuds) et les *graph embeddings* (vectorisation de graphes).

- **Node Embeddings :** Ce concept fait référence à une méthode qui consiste à représenter les caractéristiques sémantiques et structurelles de chaque élément du code, comme les variables, les fonctions et les opérateurs, sous la forme d'un vecteur dense à faible dimension. Ces vecteurs capturent les propriétés et les relations des nœuds avec leurs

2.5. Formation des embeddings (vectorisation)

voisins immédiats. Grâce à cette représentation, il est possible de conserver les relations de proximité et de similarité entre les nœuds tout en réduisant la dimensionnalité des données.

- **Graph Embeddings** : Les embeddings de graphes étendent le concept des embeddings de nœuds en représentant des graphes entiers dans un espace vectoriel. Ils saisissent les caractéristiques globales du graphe, y compris les motifs et les relations entre les nœuds. En plus de réduire la dimensionnalité et de maintenir les relations structurelles, les graph embeddings offrent également une meilleure capacité de généralisation pour les modèles d'apprentissage.

Dans le cadre de ce projet, nous avons tiré parti de ces deux techniques en adoptant une méthode inspirée de [38] et basée sur CodeBERT pour initialiser les représentations des nœuds. CodeBERT [61] est un modèle de type transformeur qui a été pré-entraîné sur des données de code, ce qui le rend très efficace pour capturer les informations syntaxiques et sémantiques présentes dans le code source.

Les nœuds du graphe sont d'abord représentés par leurs fragments de code. CodeBERT produit des *embeddings de nœuds* qui saisissent les caractéristiques syntaxiques et sémantiques de ces fragments. Cela nous permet de bénéficier des connaissances que CodeBERT a acquises sur la syntaxe et la sémantique du code grâce à son entraînement sur de larges ensembles de données de code source. En utilisant des poids pré-entraînés, CodeBERT extrait des représentations vectorielles des fragments de code associés aux nœuds. Chaque fragment est transformé en une séquence d'identifiants d'entrée et un masque d'attention, grâce au tokenizer de CodeBERT. Cette séquence est ensuite traitée par le modèle pour obtenir les embeddings. Les vecteurs de caractéristiques sont extraits des sorties du modèle et la moyenne des caractéristiques des tokens est calculée pour obtenir une représentation vectorielle finale du fragment de code correspondant au nœud.

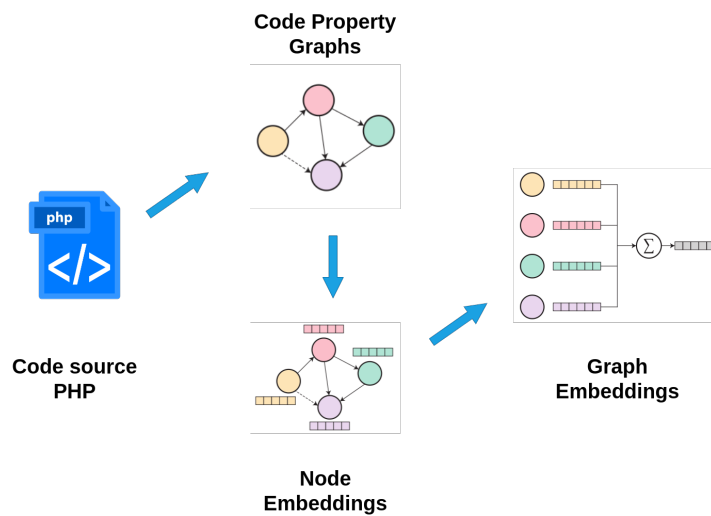


Figure 2.1. Formation des embeddings

Les relations entre les nœuds, comme les types d'arêtes et les connexions, sont par la

suite encodées pour former les *embeddings de graphes*. Ces connexions sont représentées sous forme de matrice de connectivité où chaque paire de nœuds liés est indiquée par une entrée. La structure de connectivité est encodée à l'aide d'une matrice de coordonnées qui associe chaque paire de nœuds à un indice dans le graphe. Cette représentation capture la structure du graphe et facilite l'apprentissage des relations entre les nœuds. Pour gérer les graphes de tailles différentes, nous appliquons un padding (remplissage) pour obtenir une dimension fixe limitée à 500 nœuds maximum. Cela permet de simplifier le traitement. La représentation finale du graphe est alors obtenue en combinant les embeddings de nœuds et leurs connectivités afin de produire un ensemble d'entrées prêtes pour notre modèle d'apprentissage profond.

2.6 Architecture du modèle

L'architecture du modèle que nous avons mise en place pour la détection des vulnérabilités XSS et SQLi s'appuie sur une combinaison de Graph Convolutional Networks (GCN) et de Graph Attention Networks (GAT). Cette structure tire parti des atouts des deux types de GNN et intègre également des mécanismes de Jumping Knowledge ainsi que des techniques de pooling global comme la moyenne et la somme. L'objectif est de capturer les représentations complexes des données d'entrée sous forme de graphes tout en tenant compte des relations entre les nœuds afin d'améliorer la détection des vulnérabilités XSS et SQLi. Chaque composant a été conçu pour optimiser la performance globale du modèle.

2.6.1 Description des composants

2.6.1.1 Couche GCN

Les GCN permettent de capturer la structure locale des graphes tout en prenant en compte les relations entre les nœuds voisins.

Soit un graphe non orienté $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ avec N nœuds $v_i \in \mathcal{V}$, des arêtes $(v_i, v_j) \in \mathcal{E}$, une matrice d'adjacence $A \in \mathbb{R}^{N \times N}$ (binaire ou pondérée) où $A_{ij} = 1$ si une arête existe entre les nœuds i et j et 0 sinon, une matrice de degrés $D_{ii} = \sum_j A_{ij}$. Chaque nœud $v_i \in \mathcal{V}$ possède des caractéristiques représentées par un vecteur X_i avec X la matrice des vecteurs de caractéristiques X_i . La couche de convolution d'un GCN peut être formulée comme suit :

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)} \right) \quad (2.1)$$

où :

- $H^{(l)} \in \mathbb{R}^{N \times D}$ est la matrice des activations (caractéristiques des nœuds) dans la couche l avec $H^{(0)} = X$,
- $\tilde{A} = A + I_N$ est la matrice d'adjacence du graphe non orienté $\mathcal{G}$ avec des auto-connexions ajoutées,
- I_N est la matrice d'identité,
- $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ est la matrice diagonale des degrés,
- $W^{(l)}$ est la matrice des poids à apprendre à la couche l ,
- $\sigma(\cdot)$ désigne une fonction d'activation¹ non linéaire, telle que $\text{ReLU}(\cdot) = \max(0, \cdot)$.

¹ Une fonction d'activation est une fonction mathématique utilisée dans les réseaux neuronaux pour introduire de la non-linéarité dans le modèle. Elle transforme la sortie d'un neurone et détermine si ce neurone doit être activé ou non.

2.6. Architecture du modèle

Cette couche agit comme une couche d'entrée dans notre modèle pour transformer les caractéristiques des nœuds d'entrée en une représentation de dimension plus élevée. Elle permet de capturer les interactions initiales entre les nœuds en tenant compte des relations locales dans le graphe.

2.6.1.2 Couche GAT(v2)

Les GAT intègrent un mécanisme d'attention sur les graphes et permettent d'attribuer des poids différents aux voisins d'un nœud lors de l'agrégation des informations. Soit un vecteur de caractéristiques $\mathbf{h} = \{h_1, h_2, \dots, h_N\}$, $h_i \in \mathbb{R}^F$ où N est le nombre de nœuds et F est le nombre de caractéristiques dans chaque nœud. La couche GAT produit un nouvel ensemble de caractéristiques de nœud (de cardinalité potentiellement différente F') $\mathbf{h}' = \{h'_1, h'_2, \dots, h'_N\}$, $h'_i \in \mathbb{R}^{F'}$ comme sortie [48]. Pour chaque nœud i , on calcule les coefficients d'attention e_{ij} qui indiquent l'importance des caractéristiques du nœud j pour le nœud i en utilisant une fonction d'attention :

$$e_{ij} = \text{LeakyReLU} \left(\mathbf{a}^T \left[\mathbf{W}\mathbf{x}_i \parallel \mathbf{W}\mathbf{x}_j \right] \right) \quad (2.2)$$

où :

- $\mathbf{W}$ une matrice de poids,
- $\mathbf{a}$ est un vecteur de poids d'attention,
- $\parallel$ représente l'opérateur de concaténation,
- LeakyReLU est une fonction d'activation.

Les coefficients d'attention sont ensuite normalisés à l'aide de la fonction softmax pour obtenir les coefficients d'attention normalisés α_{ij} :

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.3)$$

où $\mathcal{N}(i)$ est l'ensemble des voisins du nœud i .

Une fois obtenus, les coefficients d'attention normalisés sont utilisés pour calculer une combinaison linéaire des caractéristiques qui leur correspondent afin de servir de caractéristiques de sortie finales pour chaque nœud [48] :

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W} h_j \right) \quad (2.4)$$

où h_j est la caractéristique du nœud j et σ , une fonction d'activation non linéaire.

Les GAT peuvent également être étendus en utilisant plusieurs têtes d'attention, ce qui permet de capturer différentes relations dans le graphe. Plus précisément, K mécanismes d'attention indépendants exécutent la transformation de l'équation 2.4, puis leurs caractéristiques sont concaténées, ce qui donne la représentation suivante des caractéristiques de sortie.

$$h'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^k \mathbf{W}^k h_j \right) \quad (2.5)$$

où :

2.6. Architecture du modèle

- α_{ij}^k sont des coefficients d'attention normalisés calculés par le k -ième mécanisme d'attention (a^k),
- W^k est la matrice de poids de la transformation linéaire d'entrée correspondante.

Il faut noter que, dans ce cas, la sortie finale renvoyée h' sera composée de $K F'$ caractéristiques (plutôt que de F') pour chaque nœud.

Cependant, Brody et al. [49] ont démontré que le classement des coefficients d'attention du modèle GAT n'est pas conditionné par le nœud de requête.

Le principal inconvénient de la fonction de calcul des coefficients d'attention (voir équation 2.2) réside dans le fait que les couches apprises W et a sont appliquées de manière consécutive, ce qui permettrait de les combiner en une seule couche linéaire.

Pour résoudre ce problème, ils ont proposé le modèle GATv2 comme une version améliorée du GAT. Dans ce modèle, la couche a est appliquée après la non-linéarité (LeakyReLU), tandis que la couche W est utilisée après la concaténation. Ainsi, l'équation 2.2 se transforme en :

$$e_{ij} = a^T \text{LeakyReLU} \left(W \left[x_i \parallel x_j \right] \right) \quad (2.6)$$

Dans notre modèle, après la transformation initiale réalisée par la couche GCN, plusieurs couches GATv2 appliquent le mécanisme d'attention pour affiner les représentations des nœuds, ce qui permet au modèle de pondérer les contributions des nœuds voisins en fonction de leur importance relative. Cela permet également d'améliorer la capacité du modèle à se concentrer sur les nœuds les plus pertinents pour la tâche de détection.

2.6.1.3 Jumping Knowledge

Le Jumping Knowledge (JK) [62] est une méthode qui permet de combiner les représentations des nœuds issues de différentes couches du réseau. Cela permet de capturer à la fois les informations locales et globales présentes dans le graphe et de réduire les effets de sur-lissage. Une couche JK est définie par :

$$h_i = f \left(h_i^{(1)}, h_i^{(2)}, \dots, h_i^{(L)} \right) \quad (2.7)$$

où $h_i^{(l)}$ est la représentation du nœud i à la couche l , L le nombre total de couches et f une fonction d'agrégation.

Dans notre modèle, nous avons opté pour la concaténation comme méthode d'agrégation. Cela signifie que les sorties de toutes les couches cachées sont combinées pour créer une représentation finale. Cette méthode permet de conserver des informations issues de différentes profondeurs du réseau, ce qui améliore la capacité du modèle à capturer à la fois des caractéristiques locales et globales.

2.6.1.4 Mécanisme de pooling global

Le *pooling global* est une technique utilisée pour résumer les informations de tout un graphe en une seule représentation vectorielle. Trois méthodes de pooling global courantes sont le pooling moyen (global mean pool), le pooling par somme (global sum pool) et le pooling maximal (global max pool). Pour un graphe G composé de nœuds $\{v_1, v_2, \dots, v_N\}$, on a :

- Pooling moyen :

$$h_G^{\text{mean}} = \frac{1}{N} \sum_{i=1}^N h_i \quad (2.8)$$

- Pooling par somme :

$$h_G^{\text{sum}} = \sum_{i=1}^N h_i \quad (2.9)$$

- Pooling maximal :

$$h_G^{\text{max}} = \max_{i=1}^N h_i \quad (2.10)$$

Dans notre modèle, nous utilisons à la fois le pooling global moyen et le pooling par somme après les couches GCN et GATv2. Cette combinaison nous permet de saisir à la fois l'information moyenne et l'information totale des caractéristiques des nœuds dans le graphe.

2.6.2 Description du modèle

Notre modèle se compose principalement de deux modules : un module GNN hybride et un module de classification, chacun contenant plusieurs couches.

Module GNN hybride. La couche d'entrée reçoit les caractéristiques des nœuds du graphe issues de l'étape d'embeddings (section 2.5), représentées par une matrice $X \in \mathbb{R}^{N \times F}$ où N correspond au nombre de nœuds et F au nombre de caractéristiques par nœud. Cette matrice est ensuite transmise à la première couche de convolution qui est un GCN. Le GCN effectue une transformation linéaire suivie d'une normalisation et d'une activation non linéaire via la fonction ReLU (Rectified Linear Unit) définie par $\text{ReLU}(x) = \max(0, x)$.

Après cette première couche GCN, le modèle intègre plusieurs couches cachées de type GATv2. Chaque couche GATv2 applique un mécanisme d'attention aux voisins des nœuds, permettant de pondérer les contributions des nœuds voisins lors de la mise à jour des caractéristiques. Nous avons choisi d'utiliser un GATv2 avec 8 têtes d'attention, ce qui permet au modèle de capturer différentes relations entre les nœuds en parallèle. Les sorties de chaque tête sont ensuite concaténées et envoyées à une couche de sortie GATv2 qui génère les caractéristiques finales des nœuds. À chaque étape, un mécanisme de *dropout* est appliqué pour éviter le surapprentissage, en désactivant aléatoirement un pourcentage p des neurones durant l'entraînement.

Pour combiner les informations provenant des différentes couches, nous utilisons le mécanisme de Jumping Knowledge (JK) qui permet de concaténer les représentations des nœuds issues de chaque couche.

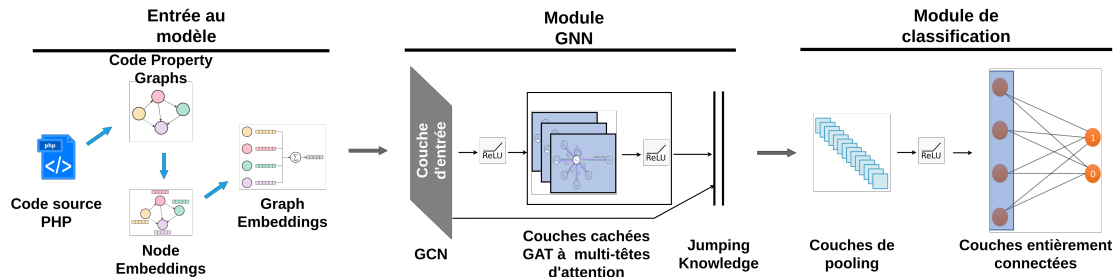


Figure 2.2. Architecture du modèle

Module de classification. Le module de classification reçoit les caractéristiques des nœuds provenant de la sortie du module GNN hybride. Ces caractéristiques sont ensuite agrégées à l'aide de deux mécanismes de pooling global : le pooling moyen et le pooling par somme. Les résultats de ces deux opérations sont ensuite concaténés pour créer une représentation qui intègre des informations globales sur le graphe. Cette représentation est ensuite envoyée à une série de couches entièrement connectées (*fully connected layers*) qui réduisent progressivement la dimensionnalité des données. La mise à jour des caractéristiques dans cette couche est décrite par :

$$x_i = \text{ReLU}(W_i x + b_i) \quad (2.11)$$

où W_i et b_i sont respectivement la matrice de poids et le biais de la couche i .

Nous avons mis en place une connexion résiduelle entre les deux premières couches entièrement connectées auxquelles nous appliquons également un mécanisme de *dropout* pour prévenir le surapprentissage. Cette connexion résiduelle contribue à atténuer le problème de la disparition du gradient dans les réseaux profonds.

La sortie de la dernière couche entièrement connectée est utilisée pour la classification finale. La couche de classification, qui est également *fully connected*, effectue une transformation linéaire pour générer les *logits* de sortie de dimension 2 représentant les classes cibles (vulnérable ou non) :

$$\text{logits} = W_{\text{classifier}} x + b_{\text{classifier}} \quad (2.12)$$

où $W_{\text{classifier}}$ et $b_{\text{classifier}}$ sont respectivement la matrice de poids et le biais de la couche de classification.

Enfin, ces *logits* sont utilisés pour prédire la probabilité d'appartenance à chaque classe, ce qui permet de déterminer si une instance donnée du graphe représente une vulnérabilité XSS ou SQLi.

2.7 Évaluation du modèle

2.7.1 Processus d'entraînement

La première étape du processus d'entraînement consiste à séparer les données. Pour notre projet, nous avons opté pour la méthode de *train-test-validation split* qui divise l'ensemble de données en trois sous-ensembles : entraînement, validation et test. Cette méthode permet d'évaluer la performance du modèle sur des données qu'il n'a pas vues lors de l'entraînement, réduisant ainsi le risque de surapprentissage. Ainsi, notre jeu de données initial est réparti en veillant à ce que les échantillons positifs et négatifs soient équilibrés dans chaque sous-ensemble. Cela garantit que le modèle apprend à généraliser plutôt qu'à simplement mémoriser les données d'entraînement.

Le processus d'apprentissage s'est ensuite déroulé sur 100 *epochs* (itérations) avec une *batch size* (taille de lots) de 128, ce qui signifie que les poids du modèle sont mis à jour après chaque groupe de 128 échantillons.

Le taux d'apprentissage (*learning rate*) est un autre hyperparamètre essentiel qui détermine l'ampleur des mises à jour des poids à chaque itération. Un taux d'apprentissage trop élevé peut provoquer une divergence du modèle, tandis qu'un taux trop bas peut ralentir la convergence. Au cours de notre entraînement, nous avons ajusté progressivement le taux

2.7. Évaluation du modèle

d'apprentissage à l'aide d'un *scheduler* de type **StepLR**(step_size, γ). Cela signifie que le taux d'apprentissage est multiplié par un facteur γ tous les step_size epochs. Cette méthode permet de commencer avec un apprentissage rapide puis de ralentir à mesure que le modèle converge.

La fonction de perte² que nous avons choisie est la Cross Entropy Loss (perte d'entropie croisée) qui est particulièrement adaptée aux problèmes de classification. Elle est définie comme suit :

$$\mathcal{L}(p, y) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (2.13)$$

où p_i représente la probabilité prédite pour la classe positive, y_i est l'étiquette réelle et N est le nombre total d'échantillons. Cette fonction pénalise les prédictions incorrectes et favorise celles qui sont correctes.

Pour l'optimisation, nous avons utilisé **AdamW**, une version améliorée de l'optimiseur Adam qui intègre une régularisation L2 sur les poids. AdamW est formulé comme suit :

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} m_t \quad (2.14)$$

où m_t est la moyenne mobile des gradients, v_t est la moyenne mobile des carrés des gradients, η est le taux d'apprentissage et ϵ est une petite constante pour éviter la division par zéro. Cet optimiseur est efficace pour traiter de grands volumes de données avec une convergence plus stable et plus rapide et aide à prévenir le sur-apprentissage en pénalisant des poids trop grands et en les ajustant en fonction des moments de premier et de second ordre.

2.7.2 Métriques d'évaluation

Pour évaluer les performances de notre modèle de détection des vulnérabilités XSS et SQLi, nous avons utilisé plusieurs métriques afin d'assurer une évaluation complète. Les principales métriques retenues sont l'exactitude (accuracy), la précision (precision), le rappel (recall), le score F1 (F1-score) et l'AUC-ROC.

1. Exactitude (Accuracy). L'exactitude mesure la proportion de prédictions correctes parmi toutes les prédictions effectuées. Cette métrique est définie par la formule :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.15)$$

où TP (True Positives) représente le nombre de vrais positifs, TN (True Negatives) le nombre de vrais négatifs, FP (False Positives) le nombre de faux positifs et FN (False Negatives) le nombre de faux négatifs.

L'exactitude est une mesure pertinente car elle reflète globalement la performance du modèle sur les classes positives et négatives.

² Une fonction de perte, également appelée fonction de coût, est utilisée pour évaluer la performance d'un modèle en mesurant l'écart entre les prédictions du modèle et les valeurs réelles.

2. Précision (Precision). La précision évalue la proportion de prédictions positives correctes parmi l'ensemble des prédictions positives réalisées par le modèle. Elle est calculée selon la formule suivante :

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.16)$$

Cette métrique est utile pour minimiser les faux positifs, ce qui est essentiel dans des scénarii où une classification incorrecte peut avoir des conséquences critiques.

3. Rappel (Recall). Le rappel (ou sensibilité) mesure la proportion de vrais positifs correctement identifiés parmi tous les cas réellement positifs. La formule est donnée par :

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.17)$$

Le rappel est important pour réduire le nombre de faux négatifs, garantissant que le modèle ne manque pas les cas critiques.

4. Score F1 (F1-Score). Le score F1 est la moyenne harmonique de la précision et du rappel. Il offre un équilibre entre ces deux métriques et est exprimé par :

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.18)$$

5. AUC-ROC (Area Under the Receiver Operating Characteristic Curve). Cette métrique évalue la capacité du modèle à distinguer entre les classes positives et négatives. La courbe ROC trace le taux de vrais positifs (TPR) contre le taux de faux positifs (FPR) à différents seuils de classification. L'AUC représente l'aire sous cette courbe et varie entre 0 et 1 où 1 indique une séparation parfaite entre les classes et 0.5 indique une performance équivalente à un tirage aléatoire. La formule pour le TPR et le FPR est la suivante :

$$\text{TPR} = \text{Recall} = \frac{TP}{TP + FN} \quad \text{et} \quad \text{FPR} = \frac{FP}{FP + TN} \quad (2.19)$$

L'AUC est utile dans des situations où il est essentiel de comprendre la capacité du modèle à classer correctement les exemples positifs par rapport aux négatifs.

Le choix de ces métriques se justifie par la nature du problème de classification binaire et par l'équilibre parfait de notre jeu de données. Dans ce contexte, l'exactitude demeure une mesure fiable et suffisante pour évaluer la performance globale du modèle, car il n'y a pas de déséquilibre significatif entre les classes. Cette métrique reflète directement la capacité du modèle à prédire correctement les classes. Néanmoins, pour obtenir une évaluation plus complète et précise, nous avons également inclus les métriques de précision, de rappel, de score F1 et d'AUC-ROC.

2.8 Conclusion

Ce chapitre a présenté en détail la méthodologie suivie pour mener à bien notre projet. Il a couvert différents aspects clés depuis l'environnement expérimental jusqu'à l'évaluation du modèle. Ces éléments nous permettront d'analyser et d'interpréter les résultats qui seront présentés dans le chapitre suivant.

RÉSULTATS ET DISCUSSION

3.1 Introduction

Dans ce chapitre, nous présentons les résultats obtenus suite à l'entraînement et à l'évaluation du modèle de détection des vulnérabilités XSS et SQLi. Ces résultats sont ensuite analysés pour évaluer l'efficacité du modèle et pour identifier les forces et les limitations de la méthode adoptée.

3.2 Présentation des résultats

Pour entraîner le modèle que nous avons conçu, nous avons utilisé la même configuration sur chacun des ensembles de données XSS et SQLi séparément. Les hyperparamètres choisis sont les suivants :

Tableau 3.1. *Hyperparamètres utilisés pour l'entraînement*

Hyperparamètre	Valeur
Learning Rate	1e-4
Weight Decay	1.3e-6
Epochs Number	100
Batch Size	128
Step Size (StepLR)	30
Gamma (StepLR)	0.2

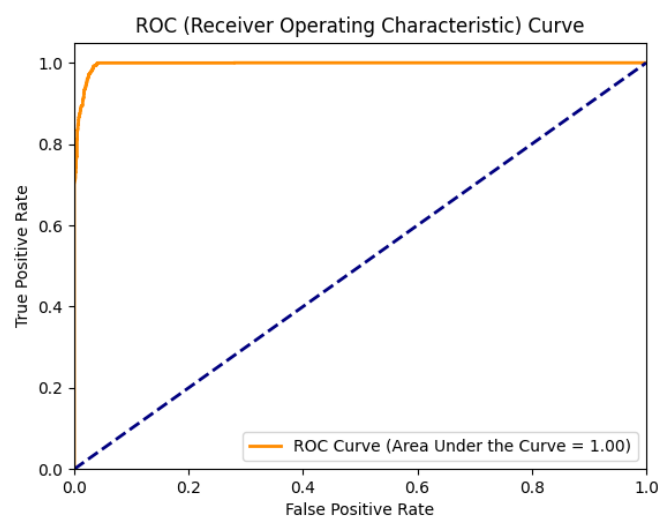
3.2.1 Résultats de la détection des vulnérabilités XSS

Les résultats obtenus lors de l'évaluation du modèle sur l'ensemble de test XSS sont assez intéressants, comme l'illustre le tableau 3.2.

Tableau 3.2. Métriques de performance (ensemble de données XSS)

Métrique	Valeur
Accuracy	97.86%
Precision	95.93%
Recall	99.96%
F1-score	97.90%
AUC-ROC	0.9965
Avg. Loss	0.0659

Le modèle présente une accuracy impressionnante de 97.86% qui indique sa capacité élevée à correctement classer les codes sources, qu'ils soient vulnérables ou non au XSS. Cette performance est confirmée par une valeur d'AUC-ROC de 0.9965, très proche de 1, indiquant une discrimination presque parfaite entre les classes vulnérables et non vulnérables (figure 3.1). Avec une précision de 95.93%, le modèle prouve sa fiabilité dans l'identification des véritables vulnérabilités XSS tout en maintenant un faible taux de faux positifs. De plus, presque toutes les vulnérabilités XSS réelles ont été identifiées, comme l'indique le rappel de 99.96%, ce qui réduit considérablement les faux négatifs. Le F1-score de 97.90% souligne également un bon équilibre optimal entre la précision et le rappel, tandis qu'une perte moyenne de 0.0659 indique que le modèle est bien ajusté et confiant dans ses prédictions.

**Figure 3.1.** Courbe ROC (ensemble de données XSS)

L'examen des courbes d'apprentissage (figure 3.2) et d'exactitude (figure 3.3) révèle une convergence rapide et stable du modèle à travers les epochs. La diminution significative de la perte tant pour l'ensemble d'entraînement que de validation suggère un apprentissage efficace et une bonne généralisation ; le modèle ne souffre donc pas de surapprentissage.

3.2. Présentation des résultats

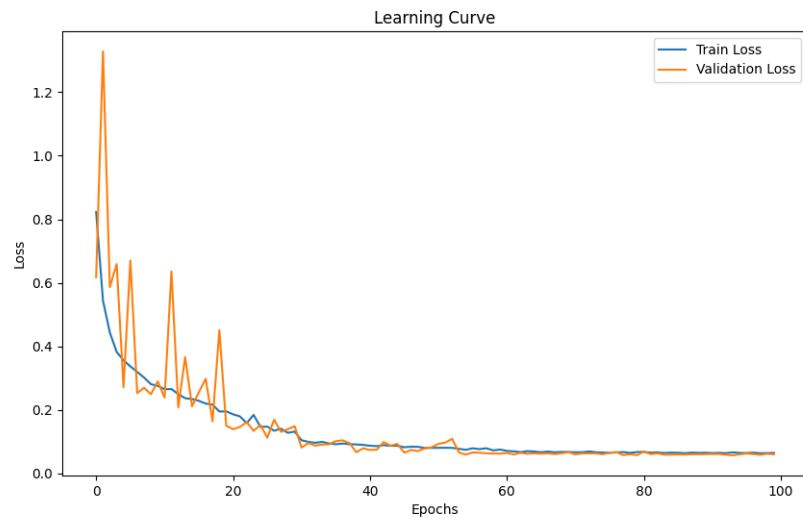


Figure 3.2. Courbe d'apprentissage (ensemble de données XSS)

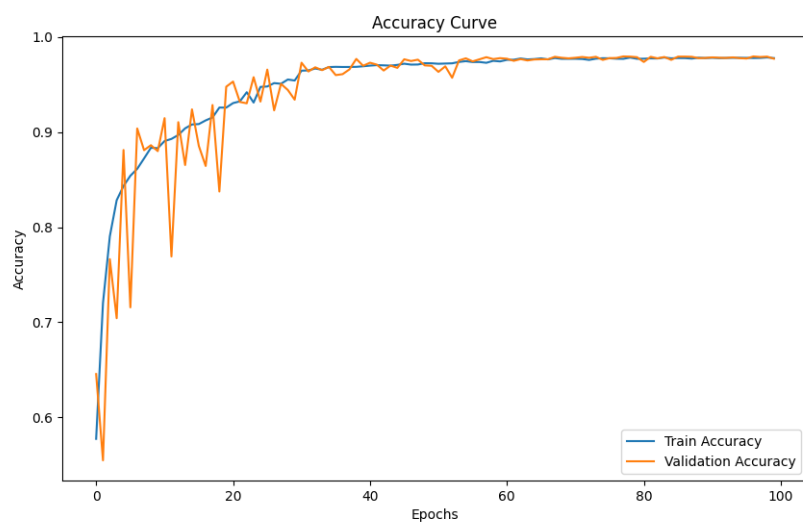


Figure 3.3. Courbe d'exactitude (ensemble de données XSS)

3.2. Présentation des résultats

L'histogramme des prédictions (figure 3.4) met en évidence une distribution bimodale distincte, avec la majorité des prédictions concentrées près de 0 et 1 indiquant une séparation claire entre les prédictions de vulnérabilité et de non-vulnérabilité. Cette observation est cohérente avec la courbe ROC (figure 3.1) qui confirme la capacité presque parfaite du modèle à distinguer les classes vulnérables des classes non vulnérables.

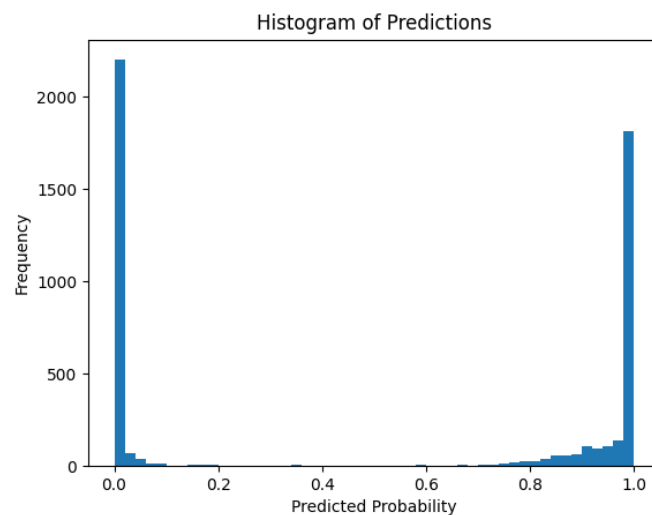


Figure 3.4. Histogramme de prédictions (ensemble de données XSS)

La matrice de confusion (figure 3.5) fournit des détails supplémentaires sur les performances du modèle. Elle montre des résultats très positifs, avec un nombre élevé de vrais positifs et de vrais négatifs. Cela indique que le modèle est performant pour détecter les vulnérabilités XSS tout en minimisant les faux positifs.

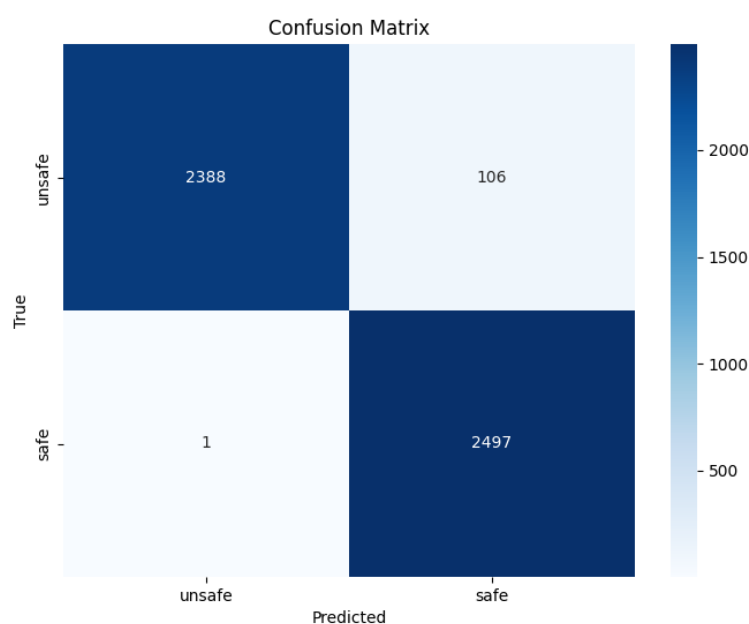


Figure 3.5. Matrice de confusion (ensemble de données XSS)

3.2.2 Résultats de la détection des vulnérabilités SQLi

Le modèle entraîné sur l'ensemble des données SQLi a donné de bons résultats, même s'ils sont légèrement en deçà de ceux obtenus pour les vulnérabilités XSS, comme le montre le tableau 3.3.

Tableau 3.3. Métriques de performance (ensemble de données SQLi)

Métrique	Valeur
Accuracy	93.10%
Precision	90.60%
Recall	96.12%
F1-score	93.28%
AUC-ROC	0.9822
Avg. Loss	0.2102

La perte moyenne de 0.2102, plus élevée que celle observée pour l'ensemble de données XSS (0.0659), indique que l'erreur résiduelle entre les prédictions du modèle et les valeurs réelles est modérée. Cela signifie que, bien que le modèle soit généralement précis, il y a des cas où il ne parvient pas à prédire correctement, ce qui se traduit par une perte plus importante. L'exactitude de 93.10% montre que le modèle identifie correctement la grande majorité des cas, qu'ils soient vulnérables ou non aux injections SQL ; et avec une précision de 90.60%, bien que légèrement inférieure, il a une bonne capacité à éviter les faux positifs, c'est-à-dire qu'il identifie correctement les cas non vulnérables au SQLi, bien qu'il laisse passer quelques vulnérabilités. Le rappel de 96.12% est élevé, indiquant que le modèle détecte presque toutes les vulnérabilités SQLi réelles. Le F1-score de 93.28% souligne que le modèle maintient une bonne performance globale dans la détection des vulnérabilités SQLi et la valeur très proche de 1 de l'AUC-ROC (0.9822) suggère que le modèle a une bonne capacité de discrimination entre les classes vulnérables et non vulnérables.

La courbe d'exactitude (figure 3.6) montre une progression stable de l'exactitude tant pour l'ensemble d'entraînement que de validation. On observe une convergence progressive des deux courbes, ce qui indique une bonne généralisation du modèle. La fluctuation plus importante de la courbe de validation par rapport à celle d'entraînement suggère une sensibilité du modèle aux variations dans les données.

3.2. Présentation des résultats

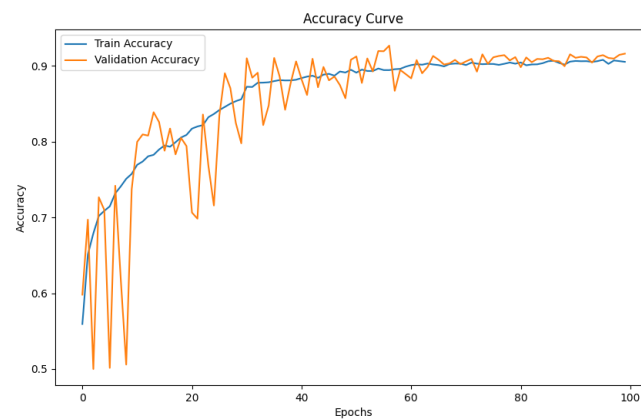


Figure 3.6. Courbe d'exactitude (ensemble de données *SQLi*)

La courbe d'apprentissage (figure 3.7) illustre la diminution de la perte pour les ensembles d'entraînement et de validation au cours des epochs. La convergence vers une perte plus faible confirme que le modèle s'adapte bien aux données.

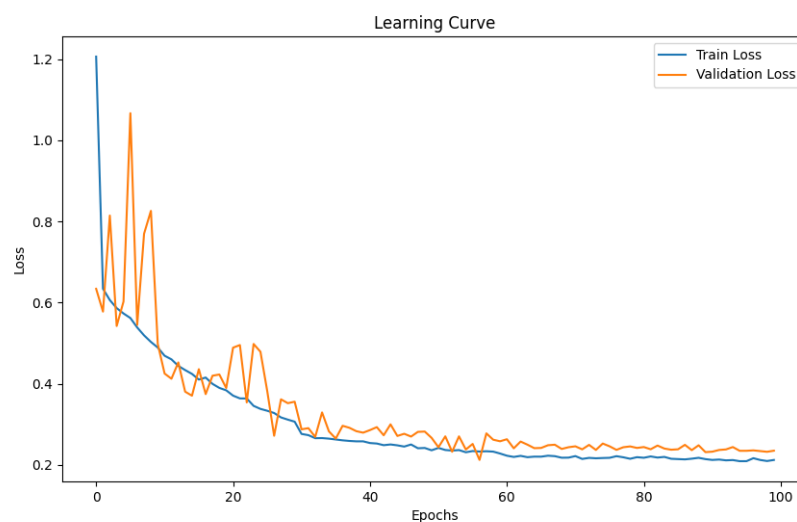


Figure 3.7. Courbe d'apprentissage (ensemble de données *SQLi*)

La courbe ROC (figure 3.8) et son AUC élevé (0.98) illustrent la capacité du modèle à distinguer efficacement entre les classes vulnérables et non vulnérables.

3.2. Présentation des résultats

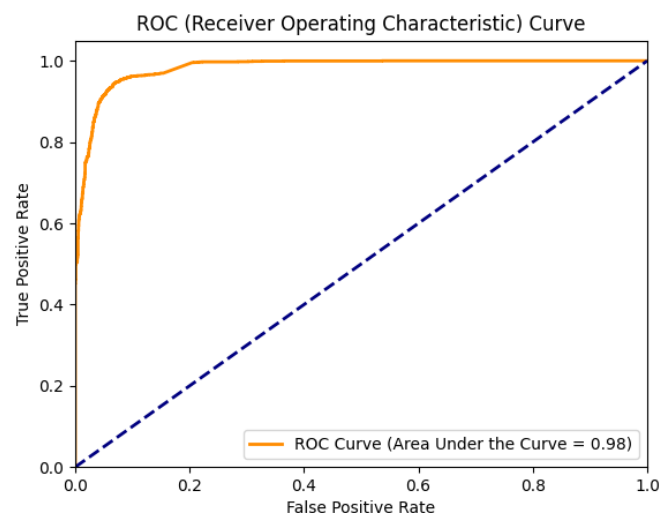


Figure 3.8. *Courbe ROC (ensemble de données SQLi)*

L'histogramme des prédictions (figure 3.9) présente une distribution bimodale avec des pics distincts aux extrémités. Cela suggère que le modèle classe généralement la plupart des codes sources soit comme clairement vulnérables, soit comme clairement non vulnérables. Cependant, la présence d'une zone intermédiaire plus importante que pour l'ensemble de données XSS indique une certaine incertitude du modèle pour certains cas, ce qui pourrait expliquer la légère baisse de performance par rapport à la détection des vulnérabilités XSS.

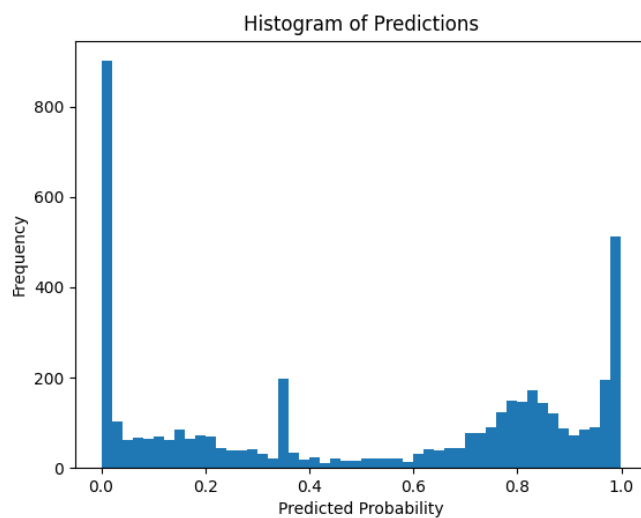


Figure 3.9. *Histogramme de prédictions (ensemble de données SQLi)*

3.3. Discussion

La matrice de confusion pour l'ensemble de données SQLi (figure 3.10) montre que le modèle a bien identifié un nombre important de vulnérabilités, mais il a également généré un certain nombre de faux positifs et a manqué quelques véritables vulnérabilités. Cela explique une fois de plus la légère baisse de performance par rapport à la détection des vulnérabilités XSS.

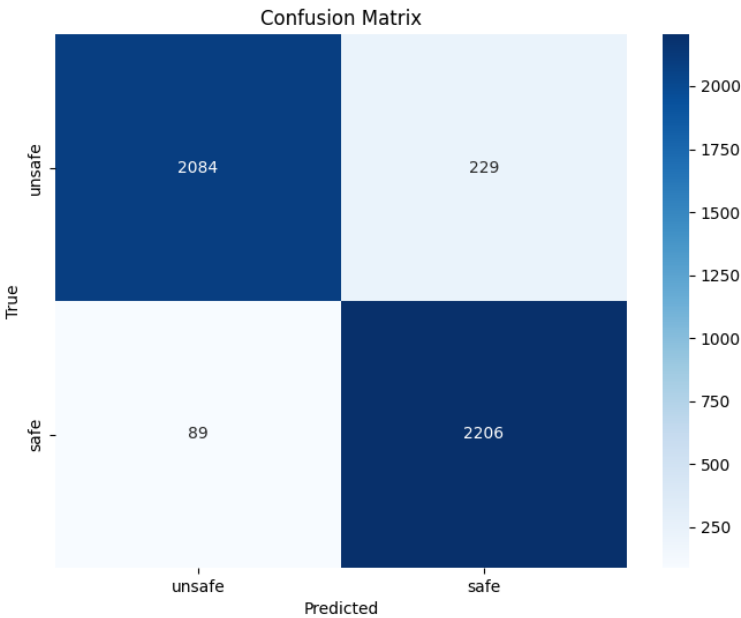


Figure 3.10. Matrice de confusion (ensemble de données SQLi)

3.3 Discussion

Une analyse comparative des résultats obtenus sur les ensembles de test XSS et SQLi révèle des performances globalement plus élevées pour la détection des vulnérabilités XSS comme l'indique le tableau 3.4.

Tableau 3.4. Comparaison des performances du modèle entre XSS et SQLi

Vulnérabilité	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC-ROC	Avg. Loss
XSS	97.86	95.93	99.96	97.90	0.9965	0.0659
SQLi	93.10	90.60	96.12	93.28	0.9822	0.2102

Ces différences de performances peuvent s'expliquer par plusieurs facteurs. Tout d'abord, bien que les ensembles de données soient équilibrés, les caractéristiques des attaques XSS et SQLi peuvent varier en termes de complexité et de diversité, ce qui influence la capacité du modèle à les apprendre et à les généraliser. La nature des vulnérabilités elles-mêmes pourrait jouer un rôle, certaines caractéristiques du code source étant plus discriminantes pour détecter les XSS que les SQLi, ou vice versa. Le choix des hyperparamètres, tels que le taux d'apprentissage, le nombre de couches, la taille des embeddings, etc, peut également avoir un impact significatif sur les performances. Un ajustement de ces paramètres peut

3.3. Discussion

permettre d'améliorer les résultats. De plus, la qualité de l'extraction des CPG et de la formation des embeddings influence directement la qualité des représentations fournies au modèle.

Pour évaluer l'efficacité de notre modèle hybride, nous l'avons comparé à trois architectures GNN populaires : GATv2, GCN et GGNN. Pour chaque architecture, nous avons utilisé deux couches de convolution et un mécanisme de pooling global moyen, tout en conservant les mêmes paramètres que ceux de notre modèle lors de l'entraînement. Les résultats présentés dans le tableau 3.5 montrent clairement la supériorité de notre modèle hybride par rapport aux implémentations standards de GATv2, GCN et GGNN, tant pour la détection des vulnérabilités XSS que SQLi.

Tableau 3.5. Performances du modèle sur différentes architectures GNN

Vulnérabilité	Modèle	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
XSS	GCN	81.35	80.21	83.26	81.71
	GATv2	90.85	89.49	92.54	90.99
	GGNN	91.09	88.10	94.99	91.42
	Notre modèle	97.86	95.93	99.96	97.90
SQLi	GCN	70.94	73.39	65.58	69.27
	GATv2	75.24	76.88	72.03	74.38
	GGNN	79.10	80.38	76.85	78.58
	Notre modèle	93.10	90.60	96.12	93.28

La performance de notre modèle réside dans son architecture hybride qui combine les couches GCN et GATv2, lui permettant de capturer à la fois les dépendances structurelles et les interactions entre les nœuds du CPG. En outre, l'intégration du Jumping Knowledge améliore la prise en compte des informations provenant de différentes couches, tandis que le mécanisme de pooling global favorise une meilleure agrégation des représentations à l'échelle du graphe.

Cependant, malgré des performances globalement excellentes, nous observons encore un certain nombre de faux positifs et de faux négatifs, particulièrement dans la détection des vulnérabilités SQLi. Puisque le modèle a été évalué sur des ensembles de test relativement restreints en taille, une évaluation sur des jeux de données plus larges et variés permettrait d'obtenir une meilleure compréhension des performances du modèle dans des conditions réelles.

De plus, bien que nous ayons bénéficié d'un bon environnement de calcul, l'implémentation et l'entraînement du modèle ont nécessité un temps de calcul considérable comme l'indique le tableau suivant.

Tableau 3.6. Temps de calcul de l'implémentation du modèle

Étape	Durée (secondes)
Extraction des CPG	557740
Formation des embeddings	2050111
Entraînement du modèle	116056

L'extraction des CPG à partir des codes sources pour capturer leur sémantique et leur structure a nécessité un temps de calcul important. Le traitement des 96824 codes sources a duré 557740 secondes, soit environ 6.5 jours. De plus, la formation des embeddings, qui

3.4. Conclusion

consiste à projeter les CPG dans un espace vectoriel pour créer les entrées du modèle GNN, a également été très gourmande en temps, nécessitant 2050111 secondes, soit environ 24 jours. Ce temps élevé s'explique par la complexité des graphes et la nécessité de calculer des représentations pour chaque nœud de manière individuelle. L'entraînement du modèle sur les deux ensembles de données XSS et SQLi a pris 116056 secondes, soit environ 32 heures. Bien que cette étape soit plus rapide que les précédentes, ce temps reste néanmoins non négligeable. Ces temps de calcul importants pourraient limiter l'utilisation du modèle dans des environnements aux ressources restreintes ou nécessitant des prédictions en temps réel.

3.4 Conclusion

Ce chapitre a présenté les résultats obtenus par le modèle de détection automatique des vulnérabilités XSS et SQLi avec des performances globalement élevées, notamment une accuracy de 97.86% pour XSS et 93.10% pour SQLi. L'analyse comparative des performances a révélé des différences significatives entre les deux types de vulnérabilités, tout en identifiant les forces et les faiblesses du modèle.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Ce mémoire a exploré l'application des Code Property Graphs (CPG) et des Graph Neural Networks (GNN) pour la détection automatique des vulnérabilités XSS et SQLi dans les applications web, en se concentrant principalement sur le langage PHP. Nous avons alors développé une méthode exploitant les CPG et les GNN pour identifier les vulnérabilités XSS et SQLi afin de répondre aux questions de recherche formulées dans la section 3. Les réponses à ces questions de recherche sont résumées ci-dessous.

- **QR1 :** *Dans quelle mesure une méthode combinant les CPG et les GNN peut-elle améliorer la détection des vulnérabilités XSS et SQLi ?*

La combinaison des Code Property Graphs (CPG) et des Graph Neural Networks (GNN) représente en effet une méthode efficace pour détecter les vulnérabilités XSS et SQLi grâce à une modélisation approfondie de la structure et de la sémantique du code. Les CPG transforment le code source en graphes interconnectés qui intègrent des éléments comme l'Abstract Syntax Tree (AST), le Control Flow Graph (CFG) et le Program Dependence Graph (PDG). Cette représentation permet de capturer non seulement la syntaxe du code, mais aussi les flux de contrôle et les dépendances de données et de contrôle, offrant ainsi une vue d'ensemble riche et contextuelle pour l'analyse de code. Elle facilite également l'identification des vulnérabilités en mettant en évidence les interactions complexes entre les différentes parties du code. Les GNN, de leur côté, utilisent cette structure graphique pour apprendre des représentations complexes des nœuds et des arêtes, ce qui leur permet de modéliser efficacement les relations entre les différents éléments du code. Grâce à leur capacité d'apprentissage, les GNN peuvent déceler des motifs et des interactions subtiles souvent révélateurs de vulnérabilités, même dans des situations complexes. En intégrant ces deux approches, notre modèle hybride, qui combine un Graph Convolutional Network (GCN) et un Graph Attention Network (GAT), parvient à identifier avec précision les failles de sécurité. Il peut analyser simultanément plusieurs dimensions du code, ce qui est essentiel pour détecter des vulnérabilités telles que les injections XSS et SQLi souvent liées à l'interaction entre différentes parties du code.

- **QR2 :** *Quelles sont les performances du modèle proposé en termes d'exactitude, de précision, de rappel et d'autres métriques d'évaluation ?*

Nous avons abordé cette question dans la section 3.2 où nous avons présenté les résultats de l'évaluation de notre modèle sur les ensembles de données XSS et SQLi. Le modèle proposé pour détecter ces vulnérabilités a montré des performances impressionnantes, atteignant une exactitude de 97.86% pour les vulnérabilités XSS et de 93.10%

pour les vulnérabilités SQLi. En termes de précision, le modèle a obtenu un score de 95.93% pour XSS et de 90.60% pour SQLi, ce qui témoigne de sa capacité à identifier correctement les vulnérabilités tout en limitant les faux positifs. Le rappel qui indique la proportion de véritables vulnérabilités détectées a atteint un remarquable 99.96% pour XSS, signifiant que presque toutes les vulnérabilités réelles ont été identifiées. Pour les vulnérabilités SQLi, le rappel s'est établi à 96.12%, mettant également en avant une détection efficace. Le score F1 qui combine précision et rappel a été de 97.90% pour XSS et de 93.28% pour SQLi, cela reflète une solide performance globale. Enfin, l'AUC-ROC qui évalue la capacité du modèle à distinguer entre les classes vulnérables et non vulnérables a atteint 0.9965 pour XSS et 0.9822 pour SQLi. Cela indique une discrimination presque parfaite dans le cas des vulnérabilités XSS et une très bonne capacité pour celles liées aux SQLi. Ces résultats soulignent l'efficacité et la robustesse de notre modèle dans la détection automatique des vulnérabilités dans les applications web.

- **QR3** : *Quels sont les défis et les limitations de la méthode proposée et comment peuvent-ils être surmontés ?*

Bien que le modèle proposé pour la détection des vulnérabilités XSS et SQLi affiche des performances élevées, plusieurs défis et limitations demeurent, comme mentionné dans la section 3.3. Tout d'abord, le temps de calcul requis pour l'extraction des Code Property Graphs (CPG) et la formation des embeddings est considérable, ce qui peut restreindre l'utilisation du modèle dans des environnements à ressources limitées ou nécessitant des prédictions en temps réel. Pour remédier à cette situation, il serait utile d'explorer des méthodes d'extraction plus rapides et efficaces, ainsi que d'optimiser les algorithmes de formation des embeddings pour réduire le temps de traitement. De plus, le modèle actuel ne précise pas les lignes exactes du code où se trouvent les vulnérabilités. Cela pourrait être amélioré en intégrant des mécanismes permettant d'identifier ces lignes spécifiques. Enfin, même si le modèle a été testé sur des ensembles de données équilibrés, une évaluation sur des jeux de données plus larges et diversifiés pourrait aider à déceler d'éventuels biais et à améliorer la généralisation du modèle. En intégrant ces améliorations, il serait possible d'accroître l'efficacité et la fiabilité du modèle dans des scénarii réels de détection de vulnérabilités.

Après avoir exploré les différentes dimensions de la détection des vulnérabilités XSS et SQLi à travers l'utilisation des Code Property Graphs et des Graph Neural Networks, il est clair que notre méthode hybride offre des résultats impressionnants tout en révélant la capacité des approches graphiques à capturer des interactions complexes au sein du code. Les performances élevées de notre modèle démontrent son potentiel pour améliorer la sécurité des applications web. Il reste néanmoins des défis à relever, notamment en ce qui concerne le temps de calcul et la précision dans la localisation des vulnérabilités. En poursuivant nos recherches pour surmonter ces limitations, nous espérons non seulement renforcer l'efficacité de notre méthode, mais aussi contribuer à l'avancement de la détection automatique des vulnérabilités dans le développement des applications web.

RÉFÉRENCES

BIBLIOGRAPHIQUES

Bibliographie

- [7] Neil Daswani et Moudy Elbayadi. “The Yahoo Breaches of 2013 and 2014”. In: *Big Breaches : Cybersecurity Lessons for Everyone*. Berkeley, CA: Apress, 2021, pp. 155–169. ISBN: 978-1-4842-6655-7. DOI: [10.1007/978-1-4842-6655-7_7](https://doi.org/10.1007/978-1-4842-6655-7_7).
- [13] Gary Wassermann et Zhendong Su. “Static detection of cross-site scripting vulnerabilities”. In: *Proceedings of the 30th international conference on Software engineering*. 2008, pp. 171–180. DOI: [10.1145/1368088.1368112](https://doi.org/10.1145/1368088.1368112).
- [14] Raounak Benabidallah. “Identification automatique des vulnérabilités de sécurité dans les systèmes logiciels”. Theses. Université de Bretagne Sud, Nov. 2020. URL: <https://theses.hal.science/tel-03326519>.
- [17] William GJ Halfond, Jeremy Viegas, Alessandro Orso, et al. “A Classification of SQL Injection Attacks and Countermeasures.” In: *International Symposium on Signals, Systems, and Electronics*. 2006. URL: <https://api.semanticscholar.org/CorpusID:5969227>.
- [18] Justin Clarke-Salt. *SQL injection attacks and defense*. Elsevier, 2012. ISBN: 978-1-59749-963-7.
- [20] Stephen W Boyd et Angelos D Keromytis. “SQLrand : Preventing SQL injection attacks”. In: *Applied Cryptography and Network Security : Second International Conference, ACNS 2004, Yellow Mountain, China, June 8-11, 2004. Proceedings 2*. Springer. 2004, pp. 292–302. DOI: [10.1007/978-3-540-24852-1_21](https://doi.org/10.1007/978-3-540-24852-1_21).
- [25] Brian Chess et Gary McGraw. “Static analysis for security”. In: *IEEE security & privacy* 2.6 (2004), pp. 76–79. DOI: [10.1109/MSP.2004.111](https://doi.org/10.1109/MSP.2004.111).
- [26] Adam Doupé, Marco Cova, et Giovanni Vigna. “Why Johnny can’t pentest : An analysis of black-box web vulnerability scanners”. In: *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer. 2010, pp. 111–131. DOI: [10.1007/978-3-642-14215-4_7](https://doi.org/10.1007/978-3-642-14215-4_7).
- [27] Nima Shiri Harzevili, Alvine Boaye Belle, Junjie Wang, Song Wang, Zhen Ming, Nachiappan Nagappan, et al. “A survey on automated software vulnerability detection using machine learning and deep learning”. In: *arXiv preprint arXiv:2306.11673* (2023). DOI: [10.48550/arXiv.2306.11673](https://doi.org/10.48550/arXiv.2306.11673). URL: <https://arxiv.org/abs/2306.11673>.
- [28] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, et Yuyi Zhong. “Vuldeepecker : A deep learning-based system for vulnerability detection”. In: *arXiv preprint arXiv:1801.01681* (2018). DOI: [10.48550/arXiv.1801.01681](https://doi.org/10.48550/arXiv.1801.01681).
- [29] Deqing Zou, Yawei Zhu, Shouhuai Xu, Zhen Li, Hai Jin, et Hengkai Ye. “Interpreting deep learning-based vulnerability detector predictions based on heuristic searching”. In: *ACM Tran-*

- sactions on Software Engineering and Methodology (TOSEM) 30.2 (2021), pp. 1–31. doi: [10.1145/3429444](https://doi.org/10.1145/3429444).
- [30] Hoa Khanh Dam, Truyen Tran, Trang Pham, Shien Wee Ng, John Grundy, et Aditya Ghose. “Automatic feature learning for predicting vulnerable software components”. In: *IEEE Transactions on Software Engineering* 47.1 (2018), pp. 67–85. doi: [10.1109/TSE.2018.2881961](https://doi.org/10.1109/TSE.2018.2881961).
 - [31] Yang Chen, Andrew E Santosa, Ang Ming Yi, Abhishek Sharma, Asankhaya Sharma, et David Lo. “A machine learning approach for vulnerability curation”. In: *Proceedings of the 17th International Conference on Mining Software Repositories*. 2020, pp. 32–42. doi: [10.1145/3379597.3387461](https://doi.org/10.1145/3379597.3387461).
 - [32] Fabian Yamaguchi, Nico Golde, Daniel Arp, et Konrad Rieck. “Modeling and discovering vulnerabilities with code property graphs”. In: *2014 IEEE symposium on security and privacy*. IEEE. 2014, pp. 590–604. doi: [10.1109/SP.2014.44](https://doi.org/10.1109/SP.2014.44).
 - [33] Alfred V. Aho, Monica S. Lam, Ravi Sethi, et Jeffrey D. Ullman. *Compilers : Principles, Techniques, and Tools (2nd Edition)*. USA: Addison-Wesley Longman Publishing Co., Inc., 2006. ISBN: 0321486811.
 - [34] Michael Backes, Konrad Rieck, Malte Skoruppa, Ben Stock, et Fabian Yamaguchi. “Efficient and Flexible Discovery of PHP Application Vulnerabilities”. In: *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. 2017, pp. 334–349. doi: [10.1109/EuroSP.2017.14](https://doi.org/10.1109/EuroSP.2017.14).
 - [35] Frances E. Allen. “Control flow analysis”. In: *SIGPLAN Not.* 5.7 (July 1970), pp. 1–19. ISSN: 0362-1340. doi: [10.1145/390013.808479](https://doi.org/10.1145/390013.808479).
 - [36] Jorrit Kronjee, Arjen Hommersom, et Harald Vranken. “Discovering software vulnerabilities using data-flow analysis and machine learning”. In: *Proceedings of the 13th international conference on availability, reliability and security*. 2018, pp. 1–10. doi: [10.1145/3230833.3230856](https://doi.org/10.1145/3230833.3230856).
 - [37] Jeanne Ferrante, Karl J. Ottenstein, et Joe D. Warren. “The program dependence graph and its use in optimization”. In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 9.3 (July 1987), pp. 319–349. ISSN: 0164-0925. doi: [10.1145/24039.24041](https://doi.org/10.1145/24039.24041).
 - [38] Ruitong Liu, Yanbin Wang, Haitao Xu, Bin Liu, Jianguo Sun, Zhenhao Guo, et Wenrui Ma. “Source Code Vulnerability Detection : Combining Code Language Models and Code Property Graphs”. In: *arXiv preprint arXiv:2404.14719* (Apr. 2024). doi: [10.48550/arXiv.2404.14719](https://doi.org/10.48550/arXiv.2404.14719). arXiv: 2404.14719 [cs.CR]. URL: <https://arxiv.org/abs/2404.14719>.
 - [39] Seongil Wi, Sijae Woo, Joyce Jiyoung Whang, et Soeul Son. “HiddenCPG : Large-Scale Vulnerable Clone Detection Using Subgraph Isomorphism of Code Property Graphs”. In: *Proceedings of the ACM Web Conference 2022. WWW ’22. Virtual Event, Lyon, France: Association for Computing Machinery, 2022*, pp. 755–766. ISBN: 9781450390965. doi: [10.1145/3485447.3512235](https://doi.org/10.1145/3485447.3512235).
 - [40] Benjamin Bowman et H Howie Huang. “VGRAPH : A Robust Vulnerable Code Clone Detection System Using Code Property Triplets”. In: *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE. 2020, pp. 53–69. doi: [10.1109/EuroSP48549.2020.00012](https://doi.org/10.1109/EuroSP48549.2020.00012).
 - [41] Ammar Ahmed, Anwar Said, Mudassir Shabbir, et Xenofon Koutsoukos. “Sequential Graph Neural Networks for Source Code Vulnerability Identification”. In: *arXiv preprint arXiv:2306.05375* (2023). doi: [10.48550/arXiv.2306.05375](https://doi.org/10.48550/arXiv.2306.05375). arXiv: 2306.05375 [cs.CR]. URL: <https://arxiv.org/abs/2306.05375>.
 - [42] Sahil Suneja, Yunhui Zheng, Yufan Zhuang, Jim Laredo, et Alessandro Morari. “Learning to map source code to software vulnerability using code-as-a-graph”. In: *arXiv preprint arXiv:2006.08614* (2020). doi: [10.48550/arXiv.2006.08614](https://doi.org/10.48550/arXiv.2006.08614). arXiv: 2006.08614 [cs.SE]. URL: <https://arxiv.org/abs/2006.08614>.

- [43] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, et Baishakhi Ray. “Deep Learning Based Vulnerability Detection : Are We There Yet?” In: *IEEE Transactions on Software Engineering* 48.9 (2021), pp. 3280–3296. doi: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).
- [45] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, et Gabriele Monfardini. “The Graph Neural Network Model”. In: *IEEE transactions on neural networks* 20.1 (2008), pp. 61–80. doi: [10.1109/TNN.2008.2005605](https://doi.org/10.1109/TNN.2008.2005605).
- [47] Thomas N Kipf et Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *arXiv preprint arXiv:1609.02907* (2016). doi: [10.48550/arXiv.1609.02907](https://doi.org/10.48550/arXiv.1609.02907). arXiv: [1609.02907](https://arxiv.org/abs/1609.02907) [cs.LG]. URL: <https://arxiv.org/abs/1609.02907>.
- [48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, et Yoshua Bengio. “Graph Attention Networks”. In: *arXiv preprint arXiv:1710.10903* (2017). doi: [10.48550/arXiv.1710.10903](https://doi.org/10.48550/arXiv.1710.10903). arXiv: [1710.10903](https://arxiv.org/abs/1710.10903) [stat.ML]. URL: <https://arxiv.org/abs/1710.10903>.
- [49] Shaked Brody, Uri Alon, et Eran Yahav. “How Attentive are Graph Attention Networks?” In: *arXiv preprint arXiv:2105.14491* (2021). doi: [10.48550/arXiv.2105.14491](https://doi.org/10.48550/arXiv.2105.14491). arXiv: [2105.14491](https://arxiv.org/abs/2105.14491) [cs.LG]. URL: <https://arxiv.org/abs/2105.14491>.
- [50] Yujia Li, Daniel Tarlow, Marc Brockschmidt, et Richard Zemel. “Gated Graph Sequence Neural Networks”. In: *arXiv preprint arXiv:1511.05493* (2015). doi: [10.48550/arXiv.1511.05493](https://doi.org/10.48550/arXiv.1511.05493). arXiv: [1511.05493](https://arxiv.org/abs/1511.05493) [cs.LG]. URL: <https://arxiv.org/abs/1511.05493>.
- [51] Will Hamilton, Zhitaoying, et Jure Leskovec. “Inductive Representation Learning on Large Graphs”. In: *Advances in neural information processing systems* 30 (2017). doi: [10.48550/arXiv.1706.02216](https://doi.org/10.48550/arXiv.1706.02216). arXiv: [1706.02216](https://arxiv.org/abs/1706.02216) [cs.SI]. URL: <https://arxiv.org/abs/1706.02216>.
- [52] Keyulu Xu, Weihua Hu, Jure Leskovec, et Stefanie Jegelka. “How Powerful are Graph Neural Networks?” In: *arXiv preprint arXiv:1810.00826* (2018). doi: [10.48550/arXiv.1810.00826](https://doi.org/10.48550/arXiv.1810.00826). arXiv: [1810.00826](https://arxiv.org/abs/1810.00826) [cs.LG]. URL: <https://arxiv.org/abs/1810.00826>.
- [53] Boris Weisfeiler et Andrei Leman. “A reduction of a graph to a canonical form and an algebra arising during this reduction”. In: *Nauchno-Tekhnicheskaya Informatsiya* 2.9 (1968), pp. 12–16. URL: https://www.itl.zcu.cz/wl2018/pdf/wl_paper_translation.pdf.
- [54] Yaqin Zhou, Shangqing Liu, Jingkai Siow, Xiaoning Du, et Yang Liu. “Devign : Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, et R. Garnett. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/49265d2447bc3bbfe9e76306ce40a31f-Paper.pdf.
- [55] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, et Yulei Sui. “DeepWukong : Statically Detecting Software Vulnerabilities Using Deep Graph Neural Network”. In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30.3 (Apr. 2021), pp. 1–33. issn: 1049-331X. doi: [10.1145/3436877](https://doi.org/10.1145/3436877). URL: <https://doi.org/10.1145/3436877>.
- [61] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. “CodeBERT : A Pre-Trained Model for Programming and Natural Languages”. In: *arXiv preprint arXiv :2002.08155* (2020). doi: [10.48550/arXiv.2002.08155](https://doi.org/10.48550/arXiv.2002.08155). arXiv: [2002.08155](https://arxiv.org/abs/2002.08155) [cs.CL]. URL: <https://arxiv.org/abs/2002.08155>.
- [62] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, et Stefanie Jegelka. “Representation learning on graphs with jumping knowledge networks”. In: *International conference on machine learning*. PMLR. 2018, pp. 5453–5462. doi: [10.48550/arXiv.1806.03536](https://doi.org/10.48550/arXiv.1806.03536). arXiv: [1806.03536](https://arxiv.org/abs/1806.03536) [cs.LG]. URL: <https://arxiv.org/abs/1806.03536>.

Webographie

- [1] W3Techs. *Usage Statistics and Market Share of PHP for Websites, November 2024*. 2024. URL: <https://w3techs.com/technologies/details/pl-php> (visité le 19/11/2024).
- [2] CWE (Common Weakness Enumeration). *CWE Top 25 Archive*. 2023. URL: <https://cwe.mitre.org/top25/archive/> (visité le 29/07/2024).
- [3] Invicti Security (Zbigniew Banach). *CWE Top 25 for 2023 : Buffer overflows, XSS, SQL injection lead the pack*. 2023. URL: <https://www.invicti.com/blog/web-security/cwe-top-25-software-weaknesses-2023-analysis-practical-use/> (visité le 29/07/2024).
- [4] Equifax. *Equifax Data Breach Settlement*. 2017. URL: <https://www.equifaxbreachsettlement.com/> (visité le 05/08/2024).
- [5] BleepingComputer (Lawrence Abrams). *200 million Twitter users' email addresses allegedly leaked online*. 2023. URL: <https://www.bleepingcomputer.com/news/security/200-million-twitter-users-email-addresses-allegedly-leaked-online/> (visité le 05/08/2024).
- [6] CPO Magazine (Alicia Hope). *Massive Trello User Data Leak : Hacker Lists 15 Million Records on a Dark Web Hacking Forum*. 2024. URL: <https://www.cpomagazine.com/cyber-security/massive-trello-user-data-leak-hacker-lists-15-million-records-on-a-dark-web-hacking-forum/> (visité le 05/08/2024).
- [8] Ponemon Institute (BobSulli). *How data breaches affect reputation and share value*. 2017. URL: <https://ponemonsullivanreport.com/2017/11/how-data-breaches-affect-reputation-and-share-value/> (visité le 02/08/2024).
- [9] IBM. *Cost of a Data Breach Report 2020*. 2020. URL: <https://www.ibm.com/security/digital-assets/cost-data-breach-report/1Cost%20of%20a%20Data%20Breach%20Report%202020.pdf> (visité le 04/08/2024).
- [10] Verizon. *Verizon 2021 Data Breach Investigations Report*. 2021. URL: <https://www.verizon.com/about/news/verizon-2021-data-breach-investigations-report> (visité le 05/08/2024).
- [11] OWASP (Open Web Application Security Project). *OWASP Top Ten*. 2021. URL: <https://owasp.org/www-project-top-ten/> (visité le 23/05/2024).
- [12] CWE (Common Weakness Enumeration). *CWE-79 : Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')*. 2024. URL: <https://cwe.mitre.org/data/definitions/79.html> (visité le 26/05/2024).
- [15] Vice (Lorenzo Franceschi-Bicchierai). *The MySpace Worm that Changed the Internet Forever*. 2015. URL: <https://www.vice.com/en/article/the-myspace-worm-that-changed-the-internet-forever/> (visité le 09/08/2024).
- [16] BBC (Jonathan Fildes). *Twitter scrambles to block worms*. 2010. URL: <https://www.bbc.com/news/technology-11382469> (visité le 09/08/2024).
- [19] UBIKA (Christine Amory). *Injection SQL : principe & exemple*. 2023. URL: <https://www.ubikasec.com/articles/sql-injection/> (visité le 26/08/2024).
- [21] Twingate Team. *Heartland Payment Systems Data Breach : What & How It Happened ?* 2024. URL: <https://www.twingate.com/blog/tips/Heartland%20Payment%20Systems-data-breach> (visité le 26/08/2024).
- [22] Dark Reading (Mathew J. Schwartz). *Sony Hacked Again, 1 Million Passwords Exposed*. 2011. URL: <https://www.darkreading.com/cyberattacks-data-breaches/sony-hacked-again-1-million-passwords-exposed> (visité le 09/08/2024).

- [23] Computerworld (Jaikumar Vijayan). *TJX data breach : At 45.6M card numbers, it's the biggest ever*. 2007. URL: <https://www.computerworld.com/article/1650590/tjx-data-breach-at-45-6m-card-numbers-it-s-the-biggest-ever.html> (visité le 09/08/2024).
- [24] Bocasay. *Preventing and reacting to SQL Injection attacks*. 2023. URL: <https://www.bocasay.com/protection-sql-injection-attack/> (visité le 26/08/2024).
- [44] DataScientest (Raphael Kassel). *Graph Neural Networks (GNN) : qu'est-ce que c'est ?* 2023. URL: <https://datascientest.com/graph-neural-networks-tout-savoir> (visité le 24/06/2024).
- [46] Daniele Grattarola. *A practical introduction to GNNs - Part 2*. 2021. URL: <https://danielegrattarola.github.io/posts/2021-03-12/gnn-lecture-part-2.html> (visité le 26/08/2024).
- [56] CWE (Common Weakness Enumeration). *CWE-89 : Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. 2024. URL: <https://cwe.mitre.org/data/definitions/89.html> (visité le 26/05/2024).
- [57] NIST (National Institute of Standards et Technology). *NIST Software Assurance Reference Dataset*. 2024. URL: <https://samate.nist.gov/SARD/> (visité le 25/05/2024).
- [58] NIST (National Institute of Standards et Technology). *Vulnerability Test Suite Generator (VTSG)*. 2024. URL: <https://github.com/usnistgov/VTSG> (visité le 02/07/2024).
- [59] Bertrand Stivalet. *PHP Vulnerability Test Suite Generator*. 2023. URL: <https://github.com/stivalet/PHP-Vuln-test-suite-generator> (visité le 02/07/2024).
- [60] Joern - *The Bug Hunter's Workbench*. 2024. URL: <https://joern.io/> (visité le 25/05/2024).